

Module 3: Deploy The Product Catalog Agent To AgentCore Runtime

This notebook moves the Product Catalog Agent from local prototype to managed runtime.

Run 03a-ground-truth-dataset.ipynb before this notebook. Section 03 expects the managed dataset manifest and local ground-truth mirror created by 03a; if those artifacts are missing, the deployment path will stop before creating production resources.

You will learn how the deployed system is assembled: IAM roles, Lambda tool backend, Cognito identity, AgentCore Gateway, RBAC interceptor, container image, AgentCore Runtime, observability, and post-deployment evaluation evidence. The goal is not just to deploy an agent, but to produce deployment evidence that later notebooks can inspect.

Step 1: Load Deployment Context

This cell loads configuration and evidence from earlier notebooks.

Use the output to confirm the AWS account, region, model ID, dataset manifest, deployment ID, and local paths. The deployment notebook expects the managed dataset manifest plus the local `postdeploy_ground_truth.json` mirror created in Section 03a. Run `03a-ground-truth-dataset.ipynb` first if those files are not present. These identifiers become the thread that connects infrastructure, runtime, traces, and evaluation results.

```
In [1]: import boto3
import json
import os
import sys
import time
import uuid
from pathlib import Path

import hashlib

from deployment_contract import (
    DATASET_MANIFEST_PATH,
    DEPLOYMENT_MANIFEST_PATH,
    POSTDEPLOY_GROUND_TRUTH_PATH,
    assert_dataset_manifest_ready,
    load_dataset_manifest,
    make_deployment_id,
    manifest_section02_summary,
)

def _find_section_dir() -> Path:
    start = Path.cwd().resolve()
    candidates = [start, *start.parents]
    for candidate in candidates:
        if candidate.name == "03-production-deployment" and (candidate / "agents").exists():
            return candidate
        child = candidate / "03-production-deployment"
        if (child / "agents").exists():
            return child
```

```

    raise FileNotFoundError("Could not locate 03-production-deployment directory")

SECTION_DIR = _find_section_dir()
REPO_ROOT = SECTION_DIR.parent
os.chdir(SECTION_DIR)
if str(SECTION_DIR) not in sys.path:
    sys.path.insert(0, str(SECTION_DIR))

# Load region from previous module or use default
try:
    %store -r REGION
    print(f"Loaded REGION from previous module: {REGION}")
except Exception:
    session = boto3.Session()
    REGION = session.region_name or "us-west-2"
    print(f"Using default region: {REGION}")

os.environ["AWS_REGION"] = REGION
os.environ["AWS_DEFAULT_REGION"] = REGION

# Workshop configuration
WORKSHOP_PREFIX = "ecommerce-workshop"
MODULE_DIR = str(SECTION_DIR)
PRODUCTS_TABLE = f"{WORKSHOP_PREFIX}-products"
MODEL_ID = "global.anthropic.claude-sonnet-4-6"

# Release-candidate metadata
DEPLOYMENT_ID = make_deployment_id()
AGENT_VERSION = f"section03-rc-{DEPLOYMENT_ID.split('-')[1]}"

# Naming conventions
LAMBDA_ROLE_NAME = f"{WORKSHOP_PREFIX}-lambda-role"
GATEWAY_ROLE_NAME = f"{WORKSHOP_PREFIX}-gateway-role"
GATEWAY_NAME = f"{WORKSHOP_PREFIX}-product-gateway"
RUNTIME_ROLE_NAME = f"{WORKSHOP_PREFIX}-runtime-role"
RUNTIME_NAME = "ecommerce_workshop_product_catalog_agent" # underscores only - AgentCore Runtime name constraint
COGNITO_POOL_NAME = f"{WORKSHOP_PREFIX}-user-pool"
ECR_REPO_NAME = f"{WORKSHOP_PREFIX}-product-catalog-agent"
DEMO_USER_SUFFIX = DEPLOYMENT_ID.split("-")[-1]

print("\nConfiguration:")
print(f"  Section directory: {SECTION_DIR}")
print(f"  Region: {REGION}")
print(f"  Products Table: {PRODUCTS_TABLE}")
print(f"  Gateway: {GATEWAY_NAME}")
print(f"  Deployment ID: {DEPLOYMENT_ID}")
print(f"  Agent Version: {AGENT_VERSION}")

```

Loaded REGION from previous module: us-east-1

Configuration:

Section directory: /workshop/03-production-deployment
Region: us-east-1
Products Table: ecommerce-workshop-products
Gateway: ecommerce-workshop-product-gateway
Deployment ID: section03-20260920T093847Z-bcb92d3f
Agent Version: section03-rc-bcb92d3f

```
In [2]: # Verify AWS credentials
sts = boto3.client("sts", region_name=REGION)
identity = sts.get_caller_identity()
ACCOUNT_ID = identity["Account"]

print(f"AWS Account: {ACCOUNT_ID}")
print(f"AWS Identity: {identity['Arn']}")

# Load and enforce Section 03a dataset lineage. The deployment should not
# continue from local mirrors or a missing DatasetClient managed dataset.
DATASET_MANIFEST = load_dataset_manifest()
assert_dataset_manifest_ready(DATASET_MANIFEST)
SECTION02_QUALITY_CONTRACT = manifest_section02_summary(DATASET_MANIFEST)
PROMPT_VERSION = SECTION02_QUALITY_CONTRACT.get("prompt_version")
TOOL_POLICY_VERSION = SECTION02_QUALITY_CONTRACT.get("tool_policy_version")

if DATASET_MANIFEST:
    print("\nDataset lineage loaded from 03a:")
    print(f"  Dataset lineage ID: {DATASET_MANIFEST.get('dataset_lineage_id')}")
    print(f"  Source Section 02 run: {SECTION02_QUALITY_CONTRACT.get('run_id')}")
    print(f"  Ground truth mirror: {POSTDEPLOY_GROUND_TRUTH_PATH}")
else:
    raise RuntimeError("Run 03a-ground-truth-dataset.ipynb before this notebook.")
```

AWS Account: 390402578846

AWS Identity: arn:aws:sts::390402578846:assumed-role/code-editor-CodeEditorInstanceBootstrapRole-aEh96Wt4p4Up/i-074b8680ee9bb9b6f

Dataset lineage loaded from 03a:

Dataset lineage ID: dataset-lineage-142db4695d5c
Source Section 02 run: section02-release_gate-20260920T070027Z-14457d0e
Ground truth mirror: /workshop/03-production-deployment/postdeploy_ground_truth.json

Step 2: Create IAM Roles

This cell creates or updates the roles needed by the deployed architecture.

Each role has a different job: Lambda accesses product data, Gateway invokes tool targets, Runtime runs the agent container, and evaluation reads evidence. Inspect the role ARNs because later resources refer to them directly.

```
In [3]: from utils import create_lambda_execution_role

iam_client = boto3.client("iam", region_name=REGION)

# DynamoDB table ARN
```

```
products_table_arn = f"arn:aws:dynamodb:{REGION}:{ACCOUNT_ID}:table/{PRODUCTS_TABLE}"

# Create Lambda execution role
print("Creating Lambda execution role...")
lambda_role_resp = create_lambda_execution_role(
    iam_client, LAMBDA_ROLE_NAME, [products_table_arn]
)
LAMBDA_ROLE_ARN = lambda_role_resp["Role"]["Arn"]
print(f"Lambda Role ARN: {LAMBDA_ROLE_ARN}")
```

Creating Lambda execution role...
 Created IAM role: ecommerce-workshop-lambda-role
 Attached DynamoDB policy to role
 Lambda Role ARN: arn:aws:iam::390402578846:role/ecommerce-workshop-lambda-role

Step 3: Deploy The Tool Backend Lambdas

This cell deploys the Lambda functions that implement the product tools and the RBAC interceptor.

The learning point is tool isolation. The agent does not directly own data access; it reaches tools through Gateway, and the interceptor uses identity claims to filter access before tool execution. After deployment, verify each Lambda has an ARN, update status, and role association. If this step fails, Gateway will not have executable tool targets.

```
In [4]: from utils import create_lambda_function

lambda_client = boto3.client("lambda", region_name=REGION)

# Environment variables for product tools Lambda
lambda_env_vars = {"PRODUCTS_TABLE_NAME": PRODUCTS_TABLE}

# Deploy Product Tools Lambda (11 tools)
print("Deploying Product Tools Lambda...")
product_tools_result = create_lambda_function(
    lambda_client,
    f"{WORKSHOP_PREFIX}-product-tools",
    LAMBDA_ROLE_ARN,
    f"{MODULE_DIR}/lambda_tools/product_tools_lambda.py",
    "product_tools_lambda.lambda_handler",
    lambda_env_vars,
    REGION,
)
PRODUCT_TOOLS_ARN = product_tools_result["function_arn"]
print(f"Product Tools ARN: {PRODUCT_TOOLS_ARN}\n")

# Deploy RBAC Interceptor Lambda
print("Deploying RBAC Interceptor Lambda...")
interceptor_result = create_lambda_function(
    lambda_client,
    f"{WORKSHOP_PREFIX}-rbac-interceptor",
    LAMBDA_ROLE_ARN,
    f"{MODULE_DIR}/lambda_tools/rbac_interceptor_lambda.py",
    "rbac_interceptor_lambda.lambda_handler",
    {}, # No env vars needed - interceptor is stateless
    REGION,
)
```

```
INTERCEPTOR_ARN = interceptor_result["function_arn"]
print(f"RBAC Interceptor ARN: {INTERCEPTOR_ARN}")
```

Deploying Product Tools Lambda...

Created Lambda function: ecommerce-workshop-product-tools

Product Tools ARN: arn:aws:lambda:us-east-1:390402578846:function:ecommerce-workshop-product-tools

Deploying RBAC Interceptor Lambda...

Created Lambda function: ecommerce-workshop-rbac-interceptor

RBAC Interceptor ARN: arn:aws:lambda:us-east-1:390402578846:function:ecommerce-workshop-rbac-interceptor

Step 4: Create Cognito Users And Groups

This cell creates the identity layer for customer and admin personas.

In the local notebook, roles were simple session fields. In the deployed system, roles come from Cognito groups embedded in JWT claims. The output should show the user pool, client, groups, and demo users used in later runtime calls.

```
In [5]: from utils import (
        get_or_create_cognito_user_pool,
        create_cognito_group,
        get_or_create_cognito_resource_server,
        get_or_create_cognito_app_client,
        get_or_create_user_app_client,
        get_or_create_cognito_domain,
        create_test_user,
        redact_email,
        sanitize_error,
    )

cognito_client = boto3.client("cognito-idp", region_name=REGION)

# 1. Create User Pool
print("Setting up Cognito User Pool...")
USER_POOL_ID = get_or_create_cognito_user_pool(cognito_client, COGNITO_POOL_NAME)

# 2. Create groups for RBAC
print("\nCreating RBAC groups...")
create_cognito_group(
    cognito_client, USER_POOL_ID, "customer", "Read-only product catalog access"
)
create_cognito_group(
    cognito_client, USER_POOL_ID, "admin", "Full product catalog access (read + write)"
)

# 3. Create resource server and M2M app client (for Gateway JWT validation)
COGNITO_RESOURCE_SERVER_ID = f"{WORKSHOP_PREFIX}-gateway-api"
SCOPES = [
    {"ScopeName": "gateway:read", "ScopeDescription": "Read access to Gateway tools"},
    {"ScopeName": "gateway:write", "ScopeDescription": "Write access to Gateway tools"},
]
get_or_create_cognito_resource_server(
    cognito_client,
    USER_POOL_ID,
```

```

    COGNITO_RESOURCE_SERVER_ID,
    f"{WORKSHOP_PREFIX} Gateway API",
    SCOPES,
)

M2M_CLIENT_NAME = f"{WORKSHOP_PREFIX}-gateway-client"
M2M_CLIENT_ID, M2M_CLIENT_SECRET = get_or_create_cognito_app_client(
    cognito_client, USER_POOL_ID, M2M_CLIENT_NAME, COGNITO_RESOURCE_SERVER_ID, SCOPES
)

# 4. Create user-facing app client (for Streamlit Login)
USER_CLIENT_NAME = f"{WORKSHOP_PREFIX}-user-client"
USER_CLIENT_ID = get_or_create_user_app_client(
    cognito_client, USER_POOL_ID, USER_CLIENT_NAME
)

# 5. Create Cognito domain for OAuth
DOMAIN_PREFIX = f"{WORKSHOP_PREFIX}-{str(uuid.uuid4())[:8]}"
get_or_create_cognito_domain(cognito_client, USER_POOL_ID, DOMAIN_PREFIX)

# 6. Create test users with group membership
print("\nCreating test users...")
CUSTOMER_EMAIL = os.environ.get(
    "SECTION03_CUSTOMER_EMAIL", f"customer+{DEMO_USER_SUFFIX}@example.com"
)
ADMIN_EMAIL = os.environ.get(
    "SECTION03_ADMIN_EMAIL", f"admin+{DEMO_USER_SUFFIX}@example.com"
)
TEST_PASSWORD = os.environ.get(
    "SECTION03_TEST_PASSWORD", f"{DEMO_USER_SUFFIX}Aa1!z9"
)

create_test_user(
    cognito_client,
    USER_POOL_ID,
    CUSTOMER_EMAIL,
    TEST_PASSWORD,
    group_name="customer",
    name="John Smith",
)

create_test_user(
    cognito_client,
    USER_POOL_ID,
    ADMIN_EMAIL,
    TEST_PASSWORD,
    group_name="admin",
    name="Alice Admin",
)

# Discovery URL for JWT validation
COGNITO_DISCOVERY_URL = f"https://cognito-idp.{REGION}.amazonaws.com/{USER_POOL_ID}/.well-known/openid-configuration"

print(f"\nCognito Configuration:")
print(f"  User Pool ID: {USER_POOL_ID}")
print(f"  Groups: customer, admin")
print(f"  M2M Client ID: {M2M_CLIENT_ID}")

```

```
print(f" User Client ID: {USER_CLIENT_ID}")
print(f" Test Customer: {redact_email(CUSTOMER_EMAIL)}")
print(f" Test Admin: {redact_email(ADMIN_EMAIL)}")
```

Setting up Cognito User Pool...

Created user pool: us-east-1_colvappaB

Creating RBAC groups...

Created Cognito group: customer

Created Cognito group: admin

Created resource server: ecommerce-workshop-gateway-api

Created app client: 5b75u1vibdbne01u5aebva2rq5

Created user app client: 3s2941acqjivbnj4o5rvetfrg9

Created Cognito domain: ecommerce-workshop-9f4c67cc

Creating test users...

Created test user: cu***@example.com

Added cu***@example.com to group: customer

Created test user: ad***@example.com

Added ad***@example.com to group: admin

Cognito Configuration:

User Pool ID: us-east-1_colvappaB

Groups: customer, admin

M2M Client ID: 5b75u1vibdbne01u5aebva2rq5

User Client ID: 3s2941acqjivbnj4o5rvetfrg9

Test Customer: cu***@example.com

Test Admin: ad***@example.com

Step 5: Create The AgentCore Gateway

This cell creates the Gateway endpoint that exposes product tools over MCP.

The key concept is controlled tool access at the network boundary. Gateway validates identity, routes MCP calls to Lambda targets, and passes requests through the RBAC interceptor before tools run. Inspect the Gateway ID, ARN, URL, status, and authentication configuration; these values become the MCP endpoint used by the runtime.

```
In [6]: from utils import create_agentcore_gateway_role, create_gateway

gateway_client = boto3.client("bedrock-agentcore-control", region_name=REGION)

# Create Gateway IAM role (needs permission to invoke both Lambda functions)
print("Creating Gateway IAM role...")
gateway_role_resp = create_agentcore_gateway_role(
    iam_client, GATEWAY_ROLE_NAME, [PRODUCT_TOOLS_ARN, INTERCEPTOR_ARN]
)
GATEWAY_ROLE_ARN = gateway_role_resp["Role"]["Arn"]
print(f"Gateway Role ARN: {GATEWAY_ROLE_ARN}")
```

Creating Gateway IAM role...

Created Gateway IAM role: ecommerce-workshop-gateway-role

Attached Lambda invoke policy to Gateway role

Gateway Role ARN: arn:aws:iam::390402578846:role/ecommerce-workshop-gateway-role

```
In [7]: # Create Gateway with JWT auth and RBAC interceptor
# allowedClients validates client_id claim in JWT access tokens
```

```

# Both M2M and user app clients are allowed (user access tokens have client_id = USER_CLIENT_ID)
auth_config = {
    "customJWTAuthorizer": {
        "allowedClients": [M2M_CLIENT_ID, USER_CLIENT_ID],
        "discoveryUrl": COGNITO_DISCOVERY_URL,
    }
}

print("Creating AgentCore Gateway with RBAC interceptor...")
gateway_response = create_gateway(
    gateway_client,
    GATEWAY_NAME,
    GATEWAY_ROLE_ARN,
    auth_config,
    "Product Catalog Gateway with RBAC interceptor",
    interceptor_lambda_arn=INTERCEPTOR_ARN, # Attach RBAC interceptor
)

if gateway_response:
    GATEWAY_ID = gateway_response["gatewayId"]
    GATEWAY_URL = gateway_response["gatewayUrl"]
    GATEWAY_ARN = (
        f"arn:aws:bedrock-agentcore:{REGION}:{ACCOUNT_ID}:gateway/{GATEWAY_ID}"
    )
    print(f"\nGateway Created:")
    print(f"  Gateway ID: {GATEWAY_ID}")
    print(f"  Gateway URL: {GATEWAY_URL}")
    print(f"  RBAC Interceptor: Attached (REQUEST + RESPONSE)")
else:
    print("ERROR: Gateway creation failed")

```

Creating AgentCore Gateway with RBAC interceptor...

Created Gateway: ecommerce-workshop-product-gateway-7zcr2df3sg

Gateway URL: <https://ecommerce-workshop-product-gateway-7zcr2df3sg.gateway.bedrock-agentcore.us-east-1.amazonaws.com/mcp>

RBAC Interceptor: attached (REQUEST + RESPONSE)

Gateway Created:

Gateway ID: ecommerce-workshop-product-gateway-7zcr2df3sg

Gateway URL: <https://ecommerce-workshop-product-gateway-7zcr2df3sg.gateway.bedrock-agentcore.us-east-1.amazonaws.com/mcp>

RBAC Interceptor: Attached (REQUEST + RESPONSE)

Step 6: Register Product Tools As A Gateway Target

This cell registers the product tool schemas with the Gateway.

The schema tells Gateway and the agent what tools exist, what inputs they accept, and how they map to Lambda operations. Review the target ID and tool count to confirm all catalog capabilities are reachable.

```

In [8]: from utils import create_lambda_gateway_target, get_product_tool_schemas

# Get all 11 tool schemas
TOOL_SCHEMAS = get_product_tool_schemas()
print(f"Registering {len(TOOL_SCHEMAS)} product tools as Gateway target:\n")
for schema in TOOL_SCHEMAS:

```

```

    print(f" - {schema['name']}: {schema['description'][:60]}...")

# Create the Lambda target
print(f"\nCreating Gateway target...")
product_target = create_lambda_gateway_target(
    gateway_client,
    GATEWAY_ID,
    "ProductTools",
    PRODUCT_TOOLS_ARN,
    TOOL_SCHEMAS,
    "Product catalog tools - 6 read tools + 5 admin tools",
)

if product_target:
    print(f"Target created: ProductTools")
    print(f"Target ID: {product_target.get('targetId')}")
else:
    print("ERROR: Failed to create Gateway target")

```

Registering 11 product tools as Gateway target:

- search_products: Search for products in the catalog using keywords and option...
- get_product_details: Get detailed information about a specific product by its ID...
- check_inventory: Check inventory availability and stock quantity for a produc...
- get_product_recommendations: Get product recommendations based on user context (e.g. a re...
- compare_products: Compare multiple products side by side (2-5 products)...
- get_return_policy: Get return policy information, optionally for a specific pro...
- create_product: Create a new product in the catalog (admin only)...
- update_product: Update an existing product's information (admin only)...
- delete_product: Soft-delete a product by marking it as discontinued (admin o...
- update_inventory: Update inventory levels for a product (admin only)...
- update_pricing: Update pricing for a product, optionally set a sale price (a...

Creating Gateway target...

Created Gateway target: ProductTools (id: QLIY7V3TPF)

Target created: ProductTools

Target ID: QLIY7V3TPF

Step 7: Test Gateway Tool Discovery

This cell connects directly to the Gateway before the agent container is deployed.

The test proves the tool surface is available and filtered by role. If customer and admin tool counts look wrong here, fix Gateway or RBAC before building the runtime image.

```

In [9]: # Wait for Gateway to be ready
print("Waiting for Gateway resources to propagate...")
time.sleep(15)

# Get an M2M OAuth token for Gateway access
from utils import get_oauth_token

SCOPE_STRING = f"{COGNITO_RESOURCE_SERVER_ID}/gateway:read {COGNITO_RESOURCE_SERVER_ID}/gateway:write"

print("Getting OAuth token from Cognito...")
token_response = get_oauth_token(

```

```

    USER_POOL_ID, M2M_CLIENT_ID, M2M_CLIENT_SECRET, SCOPE_STRING, REGION
)

if "access_token" in token_response:
    ACCESS_TOKEN = token_response["access_token"]
    print(
        f"Access token obtained (expires in {token_response.get('expires_in', 'unknown')})s"
    )
else:
    print(f"Failed to get token: {token_response}")

```

Waiting for Gateway resources to propagate...
 Getting OAuth token from Cognito..
 Access token obtained (expires in 3600s)

```

In [10]: # Test MCP connection to Gateway - List tools
from strands.tools.mcp import MCPClient
from mcp.client.streamable_http import streamablehttp_client

print("Testing MCP connection to Gateway...\n")

def create_mcp_transport():
    return streamablehttp_client(
        GATEWAY_URL, headers={"Authorization": f"Bearer {ACCESS_TOKEN}"}
    )

mcp_client = MCPClient(create_mcp_transport)

with mcp_client:
    tools = mcp_client.list_tools_sync()
    print(f"Found {len(tools)} MCP tools via Gateway:\n")
    for tool in tools:
        print(f"  - {tool.tool_name}")

print("\nGateway connection successful!")

```

Testing MCP connection to Gateway...

Found 6 MCP tools via Gateway:

- ProductTools__check_inventory
- ProductTools__compare_products
- ProductTools__get_product_details
- ProductTools__get_product_recommendations
- ProductTools__get_return_policy
- ProductTools__search_products

Gateway connection successful!

Step 8: Test RBAC With User Tokens

This cell gets Cognito tokens for the demo users and checks the role-specific tool inventory.

The important signal is the difference between customer and admin visibility. Customer tokens should expose read-only tools; admin tokens should expose both read and write tools.

```
In [11]: from utils import get_user_token

# Get tokens for both test users
print("Getting JWT tokens for test users...\n")

customer_tokens = get_user_token(
    cognito_client, USER_POOL_ID, USER_CLIENT_ID, CUSTOMER_EMAIL, TEST_PASSWORD
)

admin_tokens = get_user_token(
    cognito_client, USER_POOL_ID, USER_CLIENT_ID, ADMIN_EMAIL, TEST_PASSWORD
)

# Decode and show claims
import base64

def decode_jwt_claims(token):
    parts = token.split(".")
    payload_b64 = parts[1]
    padding = 4 - len(payload_b64) % 4
    if padding != 4:
        payload_b64 += "=" * padding
    return json.loads(base64.urlsafe_b64decode(payload_b64))

if customer_tokens.get("id_token"):
    claims = decode_jwt_claims(customer_tokens["id_token"])
    print(f"Customer JWT claims:")
    print(f"  email: {redact_email(claims.get('email', ''))}")
    print(f"  cognito:groups: {claims.get('cognito:groups', [])}")

if admin_tokens.get("id_token"):
    claims = decode_jwt_claims(admin_tokens["id_token"])
    print(f"\nAdmin JWT claims:")
    print(f"  email: {redact_email(claims.get('email', ''))}")
    print(f"  cognito:groups: {claims.get('cognito:groups', [])}")
```

Getting JWT tokens for test users...

Got tokens for user: cu***@example.com

Got tokens for user: ad***@example.com

Customer JWT claims:

email: cu***@example.com
cognito:groups: ['customer']

Admin JWT claims:

email: ad***@example.com
cognito:groups: ['admin']

The RBAC interceptor reads validated JWT claims to determine the caller role.

In the output, focus on role-derived tool visibility rather than the token itself. Tokens are used for invocation, but they should not appear in logs, traces, manifests, or notebook artifacts.

Step 9: Build And Push The Agent Container

This cell builds the runtime image and pushes it to Amazon ECR.

The image tag and digest are release evidence. The tag makes the version human-readable; the digest makes it immutable. Later manifests use both to explain exactly what code was deployed.

```
In [12]: import subprocess

def run_shell(command: str, *, error_label: str, hide_command: bool = False):
    """Run a shell command and fail the notebook cell with useful output."""
    result = subprocess.run(command, shell=True, capture_output=True, text=True)
    if result.returncode != 0:
        command_text = "<redacted>" if hide_command else command
        output = (result.stderr or result.stdout or "").strip()
        raise RuntimeError(
            f"{error_label} failed with exit code {result.returncode}.\n"
            f"Command: {command_text}\n"
            f"Output:\n{output}"
        )
    return result

def docker_image_exists(image_tag: str) -> bool:
    result = subprocess.run(
        f"docker image inspect {image_tag}",
        shell=True,
        capture_output=True,
        text=True,
    )
    return result.returncode == 0

# Create ECR repository
ecr_client = boto3.client("ecr", region_name=REGION)

print("Creating ECR repository...")
try:
    ecr_client.create_repository(
        repositoryName=ECR_REPO_NAME,
        imageScanningConfiguration={"scanOnPush": True},
        imageTagMutability="MUTABLE",
    )
    print(f"Created: {ECR_REPO_NAME}")
except ecr_client.exceptions.RepositoryAlreadyExistsException:
    print(f"Exists: {ECR_REPO_NAME}")

ECR_REGISTRY = f"{ACCOUNT_ID}.dkr.ecr.{REGION}.amazonaws.com"
IMAGE_TAG = f"rc-{DEPLOYMENT_ID.split('-')[1]}"
```

```

LOCAL_IMAGE_TAG = f"{ECR_REPO_NAME}:{IMAGE_TAG}"
CONTAINER_URI = f"{ECR_REGISTRY}/{ECR_REPO_NAME}:{IMAGE_TAG}"
LATEST_CONTAINER_URI = f"{ECR_REGISTRY}/{ECR_REPO_NAME}:latest"
print(f"Release candidate image URI: {CONTAINER_URI}")
print(f"Convenience latest URI: {LATEST_CONTAINER_URI}")

```

Creating ECR repository...

Created: ecommerce-workshop-product-catalog-agent

Release candidate image URI: 390402578846.dkr.ecr.us-east-1.amazonaws.com/eccommerce-workshop-product-catalog-agent:rc-bcb92d3f

Convenience latest URI: 390402578846.dkr.ecr.us-east-1.amazonaws.com/eccommerce-workshop-product-catalog-agent:latest

```

In [13]: # Login to ECR
print("Logging into ECR...")
login_cmd = f"aws ecr get-login-password --region {REGION} | docker login --username AWS --password-stdin {ECR_REGISTRY}"
run_shell(login_cmd, error_label="ECR login", hide_command=True)
print("ECR login successful")

```

Logging into ECR...

ECR login successful

```

In [14]: # Build Docker image (ARM64 required by AgentCore Runtime)
print("Building Docker image (ARM64)...")
print("This may take a few minutes on first build.\n")

build_cmd = f"DOCKER_BUILDKIT=1 docker build --platform linux/arm64 -t {LOCAL_IMAGE_TAG} {MODULE_DIR}/agents/"
run_shell(build_cmd, error_label="Docker image build")

if not docker_image_exists(LOCAL_IMAGE_TAG):
    raise RuntimeError(
        f"Docker build completed but local image {LOCAL_IMAGE_TAG} was not found. "
        "Re-run the ECR image tag cell and this build cell in order."
    )

DOCKER_BUILD_SUCCEEDED = True
print(f"Build successful: {LOCAL_IMAGE_TAG}")

```

Building Docker image (ARM64)...

This may take a few minutes on first build.

Build successful: ecommerce-workshop-product-catalog-agent:rc-bcb92d3f

```

In [15]: from deployment_contract import ecr_image_evidence

# Tag and push the immutable release-candidate tag plus latest for workshop convenience.
print("Pushing release-candidate image to ECR...")

if not globals().get("DOCKER_BUILD_SUCCEEDED") or not docker_image_exists(LOCAL_IMAGE_TAG):
    raise RuntimeError(
        f"Local Docker image {LOCAL_IMAGE_TAG} does not exist. "
        "Run the Docker build cell successfully before this push cell. "
        "If you reran the setup/ECR cells, the release-candidate tag may have changed."
    )

run_shell(f"docker tag {LOCAL_IMAGE_TAG} {CONTAINER_URI}", error_label="Docker release tag")
run_shell(f"docker tag {LOCAL_IMAGE_TAG} {LATEST_CONTAINER_URI}", error_label="Docker latest tag")

push_cmd = f"docker push {CONTAINER_URI}"

```

```

run_shell(push_cmd, error_label="Docker release image push")
print(f"Pushed successfully: {CONTAINER_URI}")

run_shell(f"docker push {LATEST_CONTAINER_URI}", error_label="Docker latest image push")
print(f"Updated convenience tag: {LATEST_CONTAINER_URI}")

# The helper reads ECR imageDetails[].imageDigest for immutable release evidence.
IMAGE_EVIDENCE = ecr_image_evidence(
    ecr_client,
    repository=ECR_REPO_NAME,
    tag=IMAGE_TAG,
    image_uri=CONTAINER_URI,
    latest_uri=LATEST_CONTAINER_URI,
)
print("\nImage evidence:")
print(json.dumps(IMAGE_EVIDENCE, indent=2))

```

Pushing release-candidate image to ECR...

Pushed successfully: 390402578846.dkr.ecr.us-east-1.amazonaws.com/ecommerce-workshop-product-catalog-agent:rc-bcb92d3f

Updated convenience tag: 390402578846.dkr.ecr.us-east-1.amazonaws.com/ecommerce-workshop-product-catalog-agent:latest

Image evidence:

```

{
  "repository": "ecommerce-workshop-product-catalog-agent",
  "tag": "rc-bcb92d3f",
  "image_uri": "390402578846.dkr.ecr.us-east-1.amazonaws.com/ecommerce-workshop-product-catalog-agent:rc-bcb92d3f",
  "latest_uri": "390402578846.dkr.ecr.us-east-1.amazonaws.com/ecommerce-workshop-product-catalog-agent:latest",
  "digest": "sha256:58b312dc94e92289c09b76f668f2b85399064fc964523f2c1cbb17ca9835fc2d",
  "digest_source": "ecr.describe_images"
}

```

Step 10: Create The AgentCore Runtime

This cell deploys the container image to AgentCore Runtime and enables runtime metadata.

The runtime is the managed host for the agent. It receives prompts, connects to Gateway tools, emits traces, and returns agent responses. The output should give you the runtime ARN and deployment metadata used in later tests. If a runtime with the workshop name already exists, the helper uses `update_agent_runtime` so the new image and environment replace the old deployment. This runtime also records `runtime.config_bundle_hook_version=1` when the deployed image can read AgentCore configuration bundles; Section 05 uses that marker before it allows config-bundle A/B traffic.

```

In [16]: from utils import create_agent_runtime_role

# Create Runtime IAM role with Gateway invocation permission
print("Creating AgentCore Runtime role...")
runtime_role_resp = create_agent_runtime_role(
    iam_client,
    RUNTIME_ROLE_NAME,
    gateway_arn=GATEWAY_ARN, # Permission to invoke Gateway tools
)
RUNTIME_ROLE_ARN = runtime_role_resp["Role"]["Arn"]
print(f"Runtime Role ARN: {RUNTIME_ROLE_ARN}")

```

Creating AgentCore Runtime role...
Created Runtime IAM role: ecommerce-workshop-runtime-role
Attached configuration bundle policy
Attached X-Ray tracing policy
Runtime Role ARN: arn:aws:iam::390402578846:role/eccommerce-workshop-runtime-role

```
In [17]: from utils import create_agent_runtime
from deployment_contract import build_release_metadata_env, endpoint_otel_service_name

agentcore_client = boto3.client("bedrock-agentcore-control", region_name=REGION)

# Use the AgentCore endpoint service name so app spans, endpoint UI, online evaluation, and A/B scoring align.
OTEL_SERVICE_NAME = endpoint_otel_service_name(RUNTIME_NAME)

release_metadata_env = build_release_metadata_env(
    deployment_id=DEPLOYMENT_ID,
    agent_version=AGENT_VERSION,
    model_id=MODEL_ID,
    otel_service_name=OTEL_SERVICE_NAME,
    prompt_version=PROMPT_VERSION,
    tool_policy_version=TOOL_POLICY_VERSION,
)

# Agent environment variables – includes OTEL configuration for observability
agent_env_vars = {
    # Application configuration
    "AGENT_REGION": REGION,
    "GATEWAY_URL": GATEWAY_URL,
    "CONFIG_BUNDLE_HOOK_VERSION": "1",
    **release_metadata_env,
    # OpenTelemetry configuration for CloudWatch GenAI Observability
    "AGENT_OBSERVABILITY_ENABLED": "true",
    "OTEL_PYTHON_DISTRO": "aws_distro",
    "OTEL_PYTHON_CONFIGURATOR": "aws_configurator",
    "OTEL_TRACES_EXPORTER": "otlp",
    "OTEL_LOGS_EXPORTER": "otlp",
    "OTEL_METRICS_EXPORTER": "none",
    "OTEL_PYTHON_DISABLED_INSTRUMENTATIONS": "httpx",
    "OTEL_EXPORTER_OTLP_PROTOCOL": "http/protobuf",
    "OTEL_EXPORTER_OTLP_TRACES_ENDPOINT": f"https://xray.{REGION}.amazonaws.com/v1/traces",
    "OTEL_EXPORTER_OTLP_LOGS_ENDPOINT": f"https://logs.{REGION}.amazonaws.com/v1/logs",
    "OTEL_SEMCONV_STABILITY_OPT_IN": "gen_ai_tool_definitions",
    "OTEL_PYTHON_HTTPX_EXCLUDED_URLS": r"*.ecommerce-workshop-product-gateway.*",
    # Logging Levels (set to DEBUG for troubleshooting)
    "LOG_LEVEL": "INFO",
    "OTEL_LOG_LEVEL": "WARNING",
}

print("Agent Environment Variables:")
print("  Application:")
print(f"    AGENT_REGION: {REGION}")
print(f"    GATEWAY_URL: {GATEWAY_URL[:60]}...")
print(f"    MODEL_ID: {agent_env_vars['MODEL_ID']}")
print(f"    DEPLOYMENT_ID: {DEPLOYMENT_ID}")
print(f"    AGENT_VERSION: {AGENT_VERSION}")
print(f"    CONFIG_BUNDLE_HOOK_VERSION: 1")
```

```

print(f"    PROMPT_VERSION: {PROMPT_VERSION or 'not provided'}")
print(f"    TOOL_POLICY_VERSION: {TOOL_POLICY_VERSION or 'not provided'}")
print("\n OpenTelemetry:")
print(f"    OTEL_SERVICE_NAME: {OTEL_SERVICE_NAME}")
print(f"    OTEL_TRACES_EXPORTER: otlp -> X-Ray ({REGION})")
print(f"    OTEL_LOGS_EXPORTER: otlp -> CloudWatch Logs ({REGION})")
print("    Custom spans: runtime invocation, JWT role extraction, Gateway MCP, tool discovery, RBAC filtering, agent invocation")
print("\n Logging:")
print(f"    LOG_LEVEL: {agent_env_vars['LOG_LEVEL']}")
print(f"    OTEL_LOG_LEVEL: {agent_env_vars['OTEL_LOG_LEVEL']}")

print("\n\nDeploying Product Catalog Agent to AgentCore Runtime...")
print("This may take several minutes while the container starts up.\n")

runtime_response = create_agent_runtime(
    agentcore_client,
    runtime_name=RUNTIME_NAME,
    role_arn=RUNTIME_ROLE_ARN,
    container_uri=CONTAINER_URI,
    environment_vars=agent_env_vars,
    description=f"Product Catalog Agent release candidate {AGENT_VERSION} with RBAC and OpenTelemetry observability",
)

if runtime_response:
    RUNTIME_ID = runtime_response["agentRuntimeId"]
    RUNTIME_ARN = runtime_response["agentRuntimeArn"]
    print("\nRuntime deployed successfully!")
    print(f"    Runtime ID: {RUNTIME_ID}")
    print(f"    Runtime ARN: {RUNTIME_ARN}")
    print(f"    Status: {runtime_response.get('status')}")
    print("    Observability: Enabled (OTEL traces + GenAI events + custom release spans)")
else:
    print("ERROR: Runtime deployment failed")

```

Agent Environment Variables:

Application:

```
AGENT_REGION: us-east-1
GATEWAY_URL: https://ecommerce-workshop-product-gateway-7zcr2df3sg.gatewa...
MODEL_ID: global.anthropic.claude-sonnet-4-6
DEPLOYMENT_ID: section03-20260920T093847Z-bcb92d3f
AGENT_VERSION: section03-rc-bcb92d3f
CONFIG_BUNDLE_HOOK_VERSION: 1
PROMPT_VERSION: product-catalog-prompts-v1
TOOL_POLICY_VERSION: product-catalog-tool-policy-v1
```

OpenTelemetry:

```
OTEL_SERVICE_NAME: ecommerce_workshop_product_catalog_agent.DEFAULT
OTEL_TRACES_EXPORTER: otlp -> X-Ray (us-east-1)
OTEL_LOGS_EXPORTER: otlp -> CloudWatch Logs (us-east-1)
Custom spans: runtime invocation, JWT role extraction, Gateway MCP, tool discovery, RBAC filtering, agent invocation
```

Logging:

```
LOG_LEVEL: INFO
OTEL_LOG_LEVEL: WARNING
```

Deploying Product Catalog Agent to AgentCore Runtime...

This may take several minutes while the container starts up.

Created AgentCore Runtime: ecommerce_workshop_product_catalog_agent

Runtime ARN: arn:aws:bedrock-agentcore:us-east-1:390402578846:runtime/ecommerce_workshop_product_catalog_agent-c0VjG03avH

Waiting for runtime to be ready...

Status: CREATING...

Runtime is ready!

Runtime deployed successfully!

Runtime ID: ecommerce_workshop_product_catalog_agent-c0VjG03avH

Runtime ARN: arn:aws:bedrock-agentcore:us-east-1:390402578846:runtime/ecommerce_workshop_product_catalog_agent-c0VjG03avH

Status: READY

Observability: Enabled (OTEL traces + GenAI events + custom release spans)

Observability creates the evidence trail for a deployed agent.

OTEL traces capture agent invocation, Gateway connection, tool discovery, RBAC filtering, model calls, and tool calls. These traces are what let later notebooks inspect behavior without relying only on final text responses.

Step 10a: Verify CloudWatch Transaction Search

This cell checks whether CloudWatch Transaction Search is enabled for trace exploration.

Check whether Transaction Search is enabled, disabled, or unavailable, because that determines whether you can use the visual GenAI trace explorer or only log-based evidence. The notebook still records log-based evidence even when the visual explorer is not available.

```
In [18]: from datetime import datetime, timezone, timedelta
        from botocore.exceptions import ClientError
```

```

xray_client = boto3.client("xray", region_name=REGION)
logs_client = boto3.client("logs", region_name=REGION)

def check_transaction_search_enabled():
    """Check if CloudWatch Transaction Search is enabled by querying X-Ray."""
    print("Checking CloudWatch Transaction Search status...\n")
    try:
        end_time = datetime.now(timezone.utc)
        start_time = end_time - timedelta(hours=1)
        xray_client.get_trace_summaries(
            StartTime=start_time, EndTime=end_time, Sampling=True
        )
        print("CloudWatch Transaction Search is ENABLED")
        return True
    except ClientError as e:
        error_code = e.response["Error"]["Code"]
        if "ResourceNotFoundException" in error_code:
            print("CloudWatch Transaction Search is NOT ENABLED")
            print(f"\nTo enable it:")
            print(
                f"1. Go to: https://{REGION}.console.aws.amazon.com/cloudwatch/home?region={REGION}#xray:settings/transaction-search"
            )
            print("2. Click 'Edit'")
            print("3. Enable 'Ingest spans as structured logs in OpenTelemetry format'")
            print("4. Click 'Save'")
            return False
        elif "AccessDenied" in str(e):
            print(
                "Cannot verify (insufficient IAM permissions for X-Ray) - proceeding anyway"
            )
            return True
        else:
            print(f"Unexpected error: {error_code} - proceeding anyway")
            return True
    except Exception as e:
        print(f"Error checking status: {sanitize_error(e)} - proceeding anyway")
        return True

TRANSACTION_SEARCH_OK = check_transaction_search_enabled()

```

Checking CloudWatch Transaction Search status...

CloudWatch Transaction Search is ENABLED

Step 10b: Configure CloudWatch Log Delivery

This cell routes runtime telemetry to CloudWatch destinations.

The important output is the delivery configuration: which source emits telemetry, which destination receives it, and which log groups contain runtime and span evidence.

```

In [19]: def configure_log_delivery(
          runtime_id: str, runtime_arn: str, region: str, account_id: str
          ) -> dict:

```

```

"""
Configure CloudWatch Log Delivery for both APPLICATION_LOGS and TRACES.

Includes the prerequisite setup for Transaction Search:
1. CloudWatch Logs resource policy granting X-Ray write access to aws/spans
2. UpdateTraceSegmentDestination to enable CloudWatch Logs as trace destination
   (polls until ACTIVE before proceeding)
3. APPLICATION_LOGS delivery (GenAI events -> CWL vended log group)
4. TRACES delivery (spans -> X-Ray via aws/spans log group)

Returns dict with delivery configuration results.
"""
import time as _time

# Extract short suffix for delivery names (must be under 60 chars)
short_suffix = runtime_id.split("-")[-1] if "-" in runtime_id else runtime_id[-10:]

print(f"Configuring CloudWatch Log Delivery for observability...")
print(f" Runtime ID: {runtime_id}")
print(f" Short suffix: {short_suffix}")

results = {}

# =====
# Step 0a: Create CloudWatch Logs resource policy for X-Ray
# =====
# X-Ray needs permission to write spans to the aws/spans log group.
# This is a prerequisite for UpdateTraceSegmentDestination.
# See: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability-configure.html
print(f"\n--- Creating CloudWatch Logs resource policy for X-Ray ---")

resource_policy_document = json.dumps(
    {
        "Version": "2012-10-17",
        "Statement": [
            {
                "Sid": "TransactionSearchXRayAccess",
                "Effect": "Allow",
                "Principal": {"Service": "xray.amazonaws.com"},
                "Action": "logs:PutLogEvents",
                "Resource": [
                    f"arn:aws:logs:{region}:{account_id}:log-group:aws/spans:*",
                    f"arn:aws:logs:{region}:{account_id}:log-group:/aws/application-signals/data:*",
                ],
                "Condition": {
                    "ArnLike": {
                        "aws:SourceArn": f"arn:aws:xray:{region}:{account_id}:*"
                    },
                    "StringEquals": {"aws:SourceAccount": account_id},
                },
            },
        ],
    },
)

try:

```

```

logs_client.put_resource_policy(
    policyName="TransactionSearchXRayAccess",
    policyDocument=resource_policy_document,
)
print(" Created resource policy: TransactionSearchXRayAccess")
print(" (Grants xray.amazonaws.com -> logs:PutLogEvents on aws/spans)")
except ClientError as e:
    if "already exists" in str(e).lower():
        print(" Resource policy already exists")
    else:
        print(f" Warning: {sanitize_error(e)}")

# =====
# Step 0b: Enable CloudWatch Logs as Trace Segment Destination
# =====
# Configure X-Ray to store trace segments in CloudWatch Logs.
# The API returns Status: PENDING | ACTIVE. We must wait for
# ACTIVE before CreateDelivery will accept XRAY destinations.
print(f"\n--- Enabling CloudWatch Logs trace segment destination ---")
trace_dest_active = False
try:
    resp = xray_client.update_trace_segment_destination(
        Destination="CloudWatchLogs"
    )
    status = resp.get("Status", "UNKNOWN")
    print(
        f" UpdateTraceSegmentDestination: Destination=CloudWatchLogs, Status={status}"
    )

    if status == "ACTIVE":
        trace_dest_active = True
    else:
        # Poll GetTraceSegmentDestination until ACTIVE
        max_wait = 120
        poll_interval = 10
        waited = 0
        while waited < max_wait:
            _time.sleep(poll_interval)
            waited += poll_interval
            try:
                check = xray_client.get_trace_segment_destination()
                status = check.get(
                    "Status", check.get("Destination", {}).get("Status", "UNKNOWN")
                )
                dest = check.get("Destination", "UNKNOWN")
                print(
                    f" Waiting for trace destination to be ACTIVE (status: {status}, waited: {waited}s)..."
                )
                if status == "ACTIVE":
                    trace_dest_active = True
                    break
            except Exception as poll_err:
                print(f" Poll error: {poll_err}")

        if trace_dest_active:
            print(" Trace segment destination is ACTIVE")

```

```

        else:
            print(
                f" Warning: Trace destination did not reach ACTIVE within {max_wait}s"
            )
            print(" TRACES delivery may fail – try re-running this cell later")
    except ClientError as e:
        error_code = e.response["Error"]["Code"]
        if "InvalidRequestException" in error_code:
            # May mean it's already set – check current state
            print(" Trace destination may already be configured, checking...")
            try:
                check = xray_client.get_trace_segment_destination()
                dest = check.get("Destination", "UNKNOWN")
                status = check.get("Status", "UNKNOWN")
                print(f" Current: Destination={dest}, Status={status}")
                trace_dest_active = status == "ACTIVE"
            except Exception:
                trace_dest_active = True # Assume OK if we can't check
        else:
            print(f" Warning: {sanitize_error(e)}")
    except Exception as e:
        print(f" Warning: {sanitize_error(e)}")

# =====
# Step 1: APPLICATION_LOGS delivery (for GenAI events)
# =====
print(f"\n--- APPLICATION_LOGS (for GenAI events) ---")

# Create vended log group
vended_log_group = f"/aws/vendedlogs/bedrock-agentcore/{runtime_id}"
log_group_arn = f"arn:aws:logs:{region}:{account_id}:log-group:{vended_log_group}"

try:
    logs_client.create_log_group(logGroupName=vended_log_group)
    print(f" Created vended log group: {vended_log_group}")
except ClientError as e:
    if e.response["Error"]["Code"] == "ResourceAlreadyExistsException":
        print(f" Vended log group already exists")
    else:
        print(f" Error creating log group: {sanitize_error(e)}")
results["log_group"] = vended_log_group

# Create logs delivery source
logs_source_name = f"product-{short_suffix}-logs-src"
try:
    logs_client.put_delivery_source(
        name=logs_source_name, resourceArn=runtime_arn, logType="APPLICATION_LOGS"
    )
    print(f" Created APPLICATION_LOGS delivery source: {logs_source_name}")
except ClientError as e:
    if "ConflictException" in str(e) or "already exists" in str(e).lower():
        print(f" Logs source already exists")
    else:
        print(f" Error: {sanitize_error(e)}")
results["logs_source"] = logs_source_name

```

```

# Create logs delivery destination (CloudWatch Log Group)
logs_dest_name = f"product-{short_suffix}-logs-dst"
try:
    logs_client.put_delivery_destination(
        name=logs_dest_name,
        deliveryDestinationType="CWL",
        deliveryDestinationConfiguration={"destinationResourceArn": log_group_arn},
    )
    print(f" Created APPLICATION_LOGS delivery destination (CWL)")
except ClientError as e:
    if "ConflictException" in str(e) or "already exists" in str(e).lower():
        print(f" Logs destination already exists")
    else:
        print(f" Error: {sanitize_error(e)}")
results["logs_destination"] = logs_dest_name

# Create logs delivery connection
try:
    dest_info = logs_client.get_delivery_destination(name=logs_dest_name)
    dest_arn = dest_info["deliveryDestination"]["arn"]
    logs_client.create_delivery(
        deliverySourceName=logs_source_name, deliveryDestinationArn=dest_arn
    )
    print(f" Created APPLICATION_LOGS delivery connection")
except ClientError as e:
    if "ConflictException" in str(e) or "already exists" in str(e).lower():
        print(f" Logs delivery connection already exists")
    else:
        print(f" Error: {sanitize_error(e)}")

# =====
# Step 2: TRACES delivery (for spans in X-Ray)
# =====
print(f"\n--- TRACES (for spans in X-Ray) ---")

if not trace_dest_active:
    print(" Skipping TRACES delivery – trace segment destination is not ACTIVE")
    print(" Re-run this cell once the destination is active")
    return results

# Create traces delivery source
traces_source_name = f"product-{short_suffix}-trace-src"
try:
    logs_client.put_delivery_source(
        name=traces_source_name, resourceArn=runtime_arn, logType="TRACES"
    )
    print(f" Created TRACES delivery source: {traces_source_name}")
except ClientError as e:
    if "ConflictException" in str(e) or "already exists" in str(e).lower():
        print(f" Traces source already exists")
    else:
        print(f" Error: {sanitize_error(e)}")
results["traces_source"] = traces_source_name

# Create traces delivery destination (X-Ray)
traces_dest_name = f"product-{short_suffix}-trace-dst"

```

```

try:
    logs_client.put_delivery_destination(
        name=traces_dest_name, deliveryDestinationType="XRAY"
    )
    print(f" Created TRACES delivery destination (XRAY)")
except ClientError as e:
    if "ConflictException" in str(e) or "already exists" in str(e).lower():
        print(f" Traces destination already exists")
    else:
        print(f" Error: {sanitize_error(e)}")
results["traces_destination"] = traces_dest_name

# Create traces delivery connection
try:
    dest_info = logs_client.get_delivery_destination(name=traces_dest_name)
    dest_arn = dest_info["deliveryDestination"]["arn"]
    logs_client.create_delivery(
        deliverySourceName=traces_source_name, deliveryDestinationArn=dest_arn
    )
    print(f" Created TRACES delivery connection")
except ClientError as e:
    if "ConflictException" in str(e) or "already exists" in str(e).lower():
        print(f" Traces delivery connection already exists")
    else:
        print(f" Error creating TRACES delivery: {sanitize_error(e)}")

return results

# Configure Log delivery for the product catalog agent runtime
LOG_DELIVERY_CONFIG = configure_log_delivery(
    RUNTIME_ID, RUNTIME_ARN, REGION, ACCOUNT_ID
)

print(f"\n{'=' * 60}")
print("LOG DELIVERY SUMMARY")
print(f"\n{'=' * 60}")
print(f" APPLICATION_LOGS -> {LOG_DELIVERY_CONFIG.get('log_group', 'N/A')}")
if "traces_source" in LOG_DELIVERY_CONFIG:
    print(f" TRACES -> X-Ray (aws/spans log group)")
else:
    print(f" TRACES -> NOT CONFIGURED (re-run this cell)")
print(f"\nObservability pipeline is configured. Traces will appear in CloudWatch")
print(f"GenAI Observability dashboard after the first agent invocation.")

```

```

Configuring CloudWatch Log Delivery for observability...
Runtime ID: ecommerce_workshop_product_catalog_agent-c0VjG03avH
Short suffix: c0VjG03avH

--- Creating CloudWatch Logs resource policy for X-Ray ---
Created resource policy: TransactionSearchXRayAccess
(Grants xray.amazonaws.com -> logs:PutLogEvents on aws/spans)

--- Enabling CloudWatch Logs trace segment destination ---
Trace destination may already be configured, checking...
Current: Destination=CloudWatchLogs, Status=ACTIVE

--- APPLICATION_LOGS (for GenAI events) ---
Created vended log group: /aws/vendedlogs/bedrock-agentcore/ecommerce_workshop_product_catalog_agent-c0VjG03avH
Created APPLICATION_LOGS delivery source: product-c0VjG03avH-logs-src
Created APPLICATION_LOGS delivery destination (CWL)
Created APPLICATION_LOGS delivery connection

--- TRACES (for spans in X-Ray) ---
Created TRACES delivery source: product-c0VjG03avH-trace-src
Created TRACES delivery destination (XRAY)
Created TRACES delivery connection

=====
LOG DELIVERY SUMMARY
=====
APPLICATION_LOGS -> /aws/vendedlogs/bedrock-agentcore/ecommerce_workshop_product_catalog_agent-c0VjG03avH
TRACES -> X-Ray (aws/spans log group)

Observability pipeline is configured. Traces will appear in CloudWatch
GenAI Observability dashboard after the first agent invocation.

```

Step 10c: Configure Optional Log Data Protection

This cell attempts to apply additional protection to runtime log groups.

The workshop uses synthetic data, but production systems should still treat prompts, responses, and identity-adjacent metadata carefully. The output records whether protection was applied, skipped, or unavailable in the account.

```

In [20]: from deployment_contract import build_data_protection_policy

APPLY_CLOUDWATCH_DATA_PROTECTION = os.environ.get("APPLY_CLOUDWATCH_DATA_PROTECTION", "false").lower() in {"1", "true", "yes"}
DATA_PROTECTION_STATUS = {
    "requested": APPLY_CLOUDWATCH_DATA_PROTECTION,
    "status": "SKIPPED",
    "target_log_groups": [],
}

runtime_log_group_candidates = [
    LOG_DELIVERY_CONFIG.get("log_group"),
    f"/aws/bedrock-agentcore/runtimes/{RUNTIME_ID}-DEFAULT",
]

runtime_log_group_candidates = [g for g in runtime_log_group_candidates if g]
DATA_PROTECTION_STATUS["target_log_groups"] = runtime_log_group_candidates

```

```

if not APPLY_CLOUDWATCH_DATA_PROTECTION:
    print("CloudWatch Logs Data Protection skipped. Set APPLY_CLOUDWATCH_DATA_PROTECTION=true to apply it.")
else:
    applied = []
    failed = []
    for log_group in runtime_log_group_candidates:
        arn = f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:{log_group}:*"
        policy = build_data_protection_policy([arn])
        policy_document = {k: v for k, v in policy.items() if k != "ResourceArn"}
        try:
            logs_client.put_data_protection_policy(
                logGroupIdentifier=arn,
                policyDocument=json.dumps(policy_document),
            )
            applied.append(log_group)
            print(f"Applied data protection policy to {log_group}")
        except Exception as e:
            failed.append({"log_group": log_group, "error_type": type(e).__name__})
            print(f"Could not apply data protection to {log_group}: {type(e).__name__}")
    DATA_PROTECTION_STATUS["status"] = "APPLIED" if applied and not failed else "PARTIAL_OR_FAILED"
    DATA_PROTECTION_STATUS["applied"] = applied
    DATA_PROTECTION_STATUS["failed"] = failed

```

CloudWatch Logs Data Protection skipped. Set APPLY_CLOUDWATCH_DATA_PROTECTION=true to apply it.

The GenAI Observability dashboard should show trace timelines after telemetry arrives.

Use the screenshot to learn what a healthy trace view looks like: session timeline, span names, service name, tool calls, and safe metadata. Compare your live view against these fields when you inspect runtime spans in CloudWatch.

Step 11: Invoke The Runtime As A Customer

This cell calls the deployed runtime using a customer identity.

Read the response and metadata together. The response shows user-visible behavior; the metadata shows role, tools available, tools used, model, deployment ID, and session ID.

```

In [21]: from utils import invoke_agent_runtime

agentcore_runtime_client = boto3.client("bedrock-agentcore", region_name=REGION)

# Collect every invocation test result so the Step 14 quality gate can
# fail honestly when the runtime is erroring (see Step 14).
INVOCATION_TEST_RESULTS = []

# Test with customer token
customer_session_id = f"customer-{str(uuid.uuid4())}"

print("=" * 60)
print("Customer Test 1: Product Search (ALLOWED)")
print("=" * 60)

result = invoke_agent_runtime(
    agentcore_runtime_client,

```

```
RUNTIME_ARN,  
customer_session_id,  
{  
    "prompt": "Search for wireless headphones under $100",  
    "bearer_token": customer_tokens.get("id_token", ""),  
    "access_token": customer_tokens.get("access_token", ""),  
    "session_id": customer_session_id,  
},  
)  
  
print(f"\nStatus: {result.get('status')}")  
print(f"Role: {result.get('metadata', {}).get('role')}")  
print(f"Tools available: {result.get('metadata', {}).get('tools_available')}")  
print(f"Tools used: {result.get('metadata', {}).get('tools_used')}")  
print(f"\nResponse:\n{result.get('response', result.get('error', 'No response'))}")  
  
INVOCATION_TEST_RESULTS.append({"name": "Customer Test 1: Product Search", "result": result})
```

```
=====
Customer Test 1: Product Search (ALLOWED)
=====
```

Status: success
Role: customer
Tools available: 6
Tools used: ['search_products']

Response:
Great news! Here's what I found:

🎧 Wireless Headphones Under \$100

Wireless Bluetooth Headphones		**Noise Canceling Earbuds**	
Product ID	PROD-001	PROD-055	
Price	\$79.99	\$149.99	
Rating	★ 4.5	★ 4.5	
In Stock	Yes	Yes	
Description	Premium over-ear headphones with 40-hour battery, active noise cancellation & memory foam cushions True wireless earbuds with adaptive noise cancellation, transparency mode & 8-hour battery per charge		

Only **one** option falls within your \$100 budget:

🎧 **Wireless Bluetooth Headphones (PROD-001) - \$79.99**

This looks like a great deal! You get:

- 40-hour battery life
- Active noise cancellation
- Comfortable memory foam ear cushions

Would you like to:

- 📄 **See full details** on the Wireless Bluetooth Headphones?
- 📦 **Check inventory/stock** availability?
- 🔍 **Compare** it with the Noise Canceling Earbuds?

```
In [22]: # Customer Test 2: Attempt admin action (SHOULD BE REFUSED)
#
# This RBAC-denial test runs in its OWN session, kept out of the Step 14
# quality-gate session: the session-level Builtin.GoalSuccessRate judge
# scores a correct refusal as "goal not achieved" (0.0), which would flag
# a WARNING on a healthy deployment. Denial correctness is asserted by the
# rbac_compliance evaluator (Modules 2 & 5), not by goal success.
customer_rbac_session_id = f"customer-rbac-{str(uuid.uuid4())}"
print("=" * 60)
print("Customer Test 2: Create Product (SHOULD BE REFUSED)")
print("=" * 60)

result = invoke_agent_runtime(
    agentcore_runtime_client,
    RUNTIME_ARN,
    customer_rbac_session_id,
```

```

    {
        "prompt": "Create a new product called Super Speaker for $299.99 in Audio category",
        "bearer_token": customer_tokens.get("id_token", ""),
        "access_token": customer_tokens.get("access_token", ""),
        "session_id": customer_rbac_session_id,
    },
)

print(f"\nStatus: {result.get('status')}")
print(f"Role: {result.get('metadata', {}).get('role')}")
print(f"Tools available: {result.get('metadata', {}).get('tools_available')}")
print(f"\nResponse:\n{result.get('response', result.get('error', 'No response'))}")
print("\n(Customer should be refused - create_product is admin only)")
INVOCATION_TEST_RESULTS.append({"name": "Customer Test 2: Create Product (refusal)", "result": result})

```

```

=====
Customer Test 2: Create Product (SHOULD BE REFUSED)
=====

```

```

Status: success
Role: customer
Tools available: 6

```

Response:

I'm sorry, but I'm unable to create new products in the catalog. Adding, modifying, or deleting products are **administrative operations** that require administrative privileges, which are beyond my capabilities as the Product Catalog Assistant.

To get a new product added, you would need to:

1. **Contact your store administrator** or catalog management team.
2. **Use the admin portal/backend system** if you have access to it.
3. **Submit a product addition request** through your internal process.

In the meantime, I *can* help you with things like:

- 🔍 **Searching** for existing audio products
- 📄 **Viewing** product details and specifications
- 📦 **Checking** inventory and stock levels
- ⚖️ **Comparing** products side by side
- 💡 **Getting recommendations** based on your preferences

Is there anything else I can help you with?

(Customer should be refused - create_product is admin only)

Step 12: Invoke The Runtime As An Admin

This cell calls the deployed runtime using an admin identity.

Use the result to confirm the admin path can access catalog-management capabilities while still producing traceable metadata and safe runtime output.

```

In [23]: admin_session_id = f"admin-{str(uuid.uuid4())}"

print("=" * 60)
print("Admin Test 1: Product Search (ALLOWED)")
print("=" * 60)

```

```

result = invoke_agent_runtime(
    agentcore_runtime_client,
    RUNTIME_ARN,
    admin_session_id,
    {
        "prompt": "Search for headphone products",
        "bearer_token": admin_tokens.get("id_token", ""),
        "access_token": admin_tokens.get("access_token", ""),
        "session_id": admin_session_id,
    },
)

print(f"\nStatus: {result.get('status')}")
print(f"Role: {result.get('metadata', {}).get('role')}")
print(f"Tools available: {result.get('metadata', {}).get('tools_available')}")
print(f"Tools used: {result.get('metadata', {}).get('tools_used')}")
print(f"\nResponse: \n{result.get('response', result.get('error', 'No response'))}")

INVOCATION_TEST_RESULTS.append({"name": "Admin Test 1: Product Search", "result": result})

```

```

=====
Admin Test 1: Product Search (ALLOWED)
=====

```

```

Status: success
Role: admin
Tools available: 11
Tools used: ['search_products']

```

```

Response:
Here are the headphone products found in the catalog:

```

Field	Details
Product ID	PROD-001
Name	Wireless Bluetooth Headphones
Price	\$79.99
Category	Audio
In Stock	Yes
Rating	4.5
Description	Premium over-ear wireless headphones with 40-hour battery life, active noise cancellation, and comfortable memory foam ear cushions.

```

Only 1 result was found. Would you like to:
- 📖 View full details for this product (PROD-001)?
- ✎ Update any product information?
- 📦 Check inventory levels?

```

Let me know how you'd like to proceed!

```

In [24]: # Admin Test 2: Set a sale price (ALLOWED for admin)
print("=" * 60)
print("Admin Test 2: Update Pricing (ALLOWED)")
print("=" * 60)

```

```

result = invoke_agent_runtime(
    agentcore_runtime_client,

```

```

    RUNTIME_ARN,
    admin_session_id,
    {
        "prompt": "Set a sale price of $99.99 for PROD-033 (regular price stays $199.99) until 2026-06-30",
        "bearer_token": admin_tokens.get("id_token", ""),
        "access_token": admin_tokens.get("access_token", ""),
        "session_id": admin_session_id,
    },
)

print(f"\nStatus: {result.get('status')}")
print(f"Tools used: {result.get('metadata', {}).get('tools_used')}")
print(f"\nResponse:\n{result.get('response', result.get('error', 'No response'))}")

INVOCATION_TEST_RESULTS.append({"name": "Admin Test 2: Update Pricing", "result": result})

```

```

=====
Admin Test 2: Update Pricing (ALLOWED)
=====

```

```

Status: success
Tools used: ['update_pricing']

```

```

Response:
The pricing for PROD-033 - Portable Bluetooth Speaker has been successfully updated! Here's a summary:

```

```

| Detail | Value |
|---|---|
| Regular Price | $199.99 |
| Sale Price | $99.99 |
| Discount | 50% off |
| Sale Ends | June 30, 2026 |

```

The regular price remains unchanged at \$199.99, and the sale price of \$99.99 will be active until the end of June 2026. Let me know if you need any further adjustments!

```

In [25]: # Admin Test 3: Clean up - discontinue the test product
print("=" * 60)
print("Admin Test 3: Delete Product (ALLOWED - soft delete)")
print("=" * 60)

result = invoke_agent_runtime(
    agentcore_runtime_client,
    RUNTIME_ARN,
    admin_session_id,
    {
        "prompt": "Discontinue product PROD-033",
        "bearer_token": admin_tokens.get("id_token", ""),
        "access_token": admin_tokens.get("access_token", ""),
        "session_id": admin_session_id,
    },
)

print(f"\nStatus: {result.get('status')}")
print(f"Tools used: {result.get('metadata', {}).get('tools_used')}")
print(f"\nResponse:\n{result.get('response', result.get('error', 'No response'))}")

```

```
INVOCATION_TEST_RESULTS.append({"name": "Admin Test 3: Delete Product", "result": result})
```

```
=====
Admin Test 3: Delete Product (ALLOWED - soft delete)
=====
```

Status: success

Tools used: ['get_product_details']

Response:

Here are the details of the product to be discontinued:

- **Product ID:** PROD-033
- **Name:** Portable Bluetooth Speaker
- **Category:** Audio
- **Price:** \$199.99
- **Stock:** 80 units (currently in stock)
- **Rating:** 4.4 ★

Are you sure you want to discontinue this product? This will mark it as discontinued (soft delete). Please confirm to proceed.

Step 13: Validate Runtime Traces

This cell queries CloudWatch for recent spans from the deployed runtime.

The goal is to prove that invocation evidence exists. Look for span counts, session IDs, custom span names, and safe deployment attributes. These traces become the raw material for online evidence and feedback mining.

```
In [26]: # Wait for traces to propagate to CloudWatch
print("Waiting 30 seconds for traces to propagate to CloudWatch...")
print("(Traces typically take 1-2 minutes to appear after invocation)")
time.sleep(30)

# Agent-level span name patterns (vs. infra noise like Lambda env, HTTP verbs)
AGENT_SPAN_KEYWORDS = {"chat", "execute_tool", "execute_event_loop_cycle",
                       "invoke_agent", "tool_call", "strands", "product_catalog"}
EXPECTED_CUSTOM_SPANS = {
    "product_catalog.runtime_invocation",
    "product_catalog.jwt_role_extraction",
    "product_catalog.gateway_mcp_connection",
    "product_catalog.tool_discovery",
    "product_catalog.rbac_tool_filtering",
    "product_catalog.agent_invocation",
}

def is_agent_span(name: str) -> bool:
    """Return True if this span name is a meaningful agent-level trace."""
    name_lower = name.lower()
    return any(kw in name_lower for kw in AGENT_SPAN_KEYWORDS)

def query_spans_log_group(time_range_minutes=30):
```

```

"""Query the aws/spans log group for OTEL spans."""
print("\nQuerying aws/spans log group for OTEL traces...")
results = {"total_spans": 0, "span_types": {}, "services": set()}

end_time = int(datetime.now(timezone.utc).timestamp() * 1000)
start_time = int(
    (datetime.now(timezone.utc) - timedelta(minutes=time_range_minutes)).timestamp()
    * 1000
)

for log_group in ["aws/spans", "aws/spans/default"]:
    try:
        streams_response = logs_client.describe_log_streams(
            logGroupName=log_group,
            orderBy="LastEventTime",
            descending=True,
            limit=10,
        )
        streams = streams_response.get("logStreams", [])
        if not streams:
            continue

        print(f" Found {len(streams)} log streams in {log_group}")

        for stream in streams:
            events_response = logs_client.get_log_events(
                logGroupName=log_group,
                logStreamName=stream["logStreamName"],
                startTime=start_time,
                endTime=end_time,
                limit=200,
            )
            for event in events_response.get("events", []):
                try:
                    span_data = json.loads(event.get("message", ""))
                    results["total_spans"] += 1
                    span_name = span_data.get("name", "unknown")
                    results["span_types"][span_name] = (
                        results["span_types"].get(span_name, 0) + 1
                    )
                    scope = span_data.get("scope", {})
                    if scope.get("name"):
                        results["services"].add(scope["name"])
                except json.JSONDecodeError:
                    pass

            if results["total_spans"] > 0:
                print(f" Found {results['total_spans']} spans in {log_group}")
                break
    except logs_client.exceptions.ResourceNotFoundException:
        pass
    except Exception as e:
        print(f" Error querying {log_group}: {sanitize_error(e)}")

return results

```

```

def query_vendedlogs_for_genai_events(runtime_id, time_range_minutes=30):
    """Query vendedlogs for GenAI events (APPLICATION_LOGS delivery)."""
    print("\nQuerying vendedlogs for GenAI events...")
    log_group = f"/aws/vendedlogs/bedrock-agentcore/{runtime_id}"

    end_time = int(datetime.now(timezone.utc).timestamp() * 1000)
    start_time = int(
        (datetime.now(timezone.utc) - timedelta(minutes=time_range_minutes)).timestamp()
        * 1000
    )

    try:
        streams_response = logs_client.describe_log_streams(
            logGroupName=log_group, orderBy="LastEventTime", descending=True, limit=5
        )
        total_events = 0
        for stream in streams_response.get("logStreams", []):
            events_response = logs_client.get_log_events(
                logGroupName=log_group,
                logStreamName=stream["logStreamName"],
                startTime=start_time,
                endTime=end_time,
                limit=100,
            )
            total_events += len(events_response.get("events", []))

        if total_events > 0:
            print(f" Found {total_events} GenAI events in {log_group}")
            return total_events
    except logs_client.exceptions.ResourceNotFoundException:
        print(f" Log group not found yet: {log_group}")
        return 0
    except Exception as e:
        print(f" Error: {sanitize_error(e)}")
        return 0

# Run validation
print("=" * 60)
print("VALIDATING OBSERVABILITY DATA IN CLOUDWATCH")
print("=" * 60)

spans_results = query_spans_log_group(time_range_minutes=30)
genai_events = query_vendedlogs_for_genai_events(RUNTIME_ID, time_range_minutes=30)

# Summary
print(f"\n{'=' * 60}")
print("OBSERVABILITY VALIDATION SUMMARY")
print(f"{'=' * 60}")

total_spans = spans_results.get("total_spans", 0)
span_types = spans_results.get("span_types", {})

# Filter to meaningful agent-level spans (skip infra noise: HTTP verbs, Lambda env, etc.)
meaningful_types = {k: v for k, v in span_types.items() if is_agent_span(k)}

```

```

print(f"\n OTEL Spans (aws/spans): {total_spans} spans total")
if meaningful_types:
    meaningful_count = sum(meaningful_types.values())
    print(f" Agent-level span types ({meaningful_count} of {total_spans} total):")
    for span_type, count in sorted(meaningful_types.items(), key=lambda x: x[1], reverse=True)[:12]:
        print(f" - {span_type}: {count}")
    custom_span_hits = {name: span_types.get(name, 0) for name in EXPECTED_CUSTOM_SPANS}
    print("\n Custom Section 03 spans:")
    for span_name, count in sorted(custom_span_hits.items()):
        status = "seen" if count else "pending"
        print(f" - {span_name}: {status} ({count})")
elif span_types:
    print(" (Only infrastructure spans found – agent-level spans may still be propagating)")

print(f"\n GenAI Events (vendedlogs): {genai_events} events")

total_data = total_spans + genai_events
print(f"\n Total Observability Data: {total_data} records")

print(f"\n View in CloudWatch GenAI Observability Dashboard:")
print(
    f" https://{REGION}.console.aws.amazon.com/cloudwatch/home?region={REGION}#genai-observability:bedrockAgentCore"
)

if total_data > 0:
    print("\nObservability is working! Traces and GenAI events are being captured.")
else:
    print("\nNo observability data found yet. Possible reasons:")
    print(" - Data may still be propagating (wait 2-3 more minutes)")
    print(" - Log delivery configuration may need time to activate")
    print(" - Check the CloudWatch console directly")

```

Waiting 30 seconds for traces to propagate to CloudWatch...
(Traces typically take 1-2 minutes to appear after invocation)

=====

VALIDATING OBSERVABILITY DATA IN CLOUDWATCH

=====

Querying aws/spans log group for OTEL traces...

Found 1 log streams in aws/spans

Found 91 spans in aws/spans

Querying vendedlogs for GenAI events...

Found 6 GenAI events in /aws/vendedlogs/bedrock-agentcore/ecommerce_workshop_product_catalog_agent-c0VjG03avH

=====

OBSERVABILITY VALIDATION SUMMARY

=====

OTEL Spans (aws/spans): 91 spans total

Agent-level span types (66 of 91 total):

- chat global.anthropic.claude-sonnet-4-6: 9
- chat: 9
- execute_event_loop_cycle: 9
- product_catalog.jwt_role_extraction: 5
- product_catalog.gateway_mcp_connection: 5
- product_catalog.tool_discovery: 5
- product_catalog.rbac_tool_filtering: 5
- invoke_agent Strands Agents: 5
- product_catalog.agent_invocation: 5
- product_catalog.runtime_invocation: 5
- execute_tool ProductTools__search_products: 2
- execute_tool ProductTools__update_pricing: 1

Custom Section 03 spans:

- product_catalog.agent_invocation: seen (5)
- product_catalog.gateway_mcp_connection: seen (5)
- product_catalog.jwt_role_extraction: seen (5)
- product_catalog.rbac_tool_filtering: seen (5)
- product_catalog.runtime_invocation: seen (5)
- product_catalog.tool_discovery: seen (5)

GenAI Events (vendedlogs): 6 events

Total Observability Data: 97 records

View in CloudWatch GenAI Observability Dashboard:

<https://us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#genai-observability:bedrockAgentCore>

Observability is working! Traces and GenAI events are being captured.

The CloudWatch Logs Insights view should show span documents for recent runtime sessions.

Use the screenshot to learn what healthy log-based trace evidence looks like: session IDs, span names, service names, timestamps, tool calls, and safe deployment metadata. These fields are what later notebooks use to connect runtime behavior to evaluation evidence.

```
In [27]: from utils import save_config

# Build agent configuration for the Local Streamlit demo.
# Do not write tokens, client secrets, or passwords to this generated config.
agent_config = {
    "runtime_arn": RUNTIME_ARN,
    "runtime_id": RUNTIME_ID,
    "runtime_name": RUNTIME_NAME,
    "gateway_id": GATEWAY_ID,
    "gateway_url": GATEWAY_URL,
    "deployment_id": DEPLOYMENT_ID,
    "agent_version": AGENT_VERSION,
    "deployment_manifest_path": str(DEPLOYMENT_MANIFEST_PATH.relative_to(REPO_ROOT)),
    "demo_user_suffix": DEMO_USER_SUFFIX,
    "dataset_manifest_path": str(DATASET_MANIFEST_PATH.relative_to(REPO_ROOT)),
    "region": REGION,
    "user_pool_id": USER_POOL_ID,
    "user_client_id": USER_CLIENT_ID,
    "model_id": MODEL_ID,
    "otel_service_name": OTEL_SERVICE_NAME,
    "prompt_version": PROMPT_VERSION,
    "tool_policy_version": TOOL_POLICY_VERSION,
}

save_config(agent_config, "streamlit_app/agent_config.json")

print("\nConfiguration saved for Streamlit app.")
print("Demo login uses synthetic workshop users derived from deployment metadata; passwords are not persisted in this config.")
print("\nTo launch the chat interface, run the below commands in a terminal:")
print("  cd 03-production-deployment/streamlit_app")
print("  pip install streamlit boto3")
print("  streamlit run app.py")
print("\nThe app will be available at http://localhost:8501")
```

Saved config to: streamlit_app/agent_config.json

Configuration saved for Streamlit app.

Demo login uses synthetic workshop users derived from deployment metadata; passwords are not persisted in this config.

To launch the chat interface, run the below commands in a terminal:

```
cd 03-production-deployment/streamlit_app
pip install streamlit boto3
streamlit run app.py
```

The app will be available at http://localhost:8501

Step 14: Run Grounded Post-Deployment Evaluation

This cell invokes the deployed runtime against the prepared ground-truth scenarios and evaluates the results.

The key concept is grounded evaluation: each scenario in `postdeploy_ground_truth.json` has expectations, such as required facts, refusal behavior, or tool trajectory. The output should show scenario status, evaluator scores, trace availability, and the quality-gate summary.

If the gate does not pass, read the failure rows first. In production you would stop and fix the issue or run a formal human review. For this workshop, the next optional cell can mark the gate as passed so you can continue through the remaining mechanics of the notebook.

```
In [28]: from datetime import timedelta

from bedrock_agentcore.evaluation import EvaluationClient, ReferenceInputs
from deployment_contract import (
    evaluator_ids_for_scenario,
    load_postdeploy_ground_truth,
    reference_inputs_kwargs,
    scenario_runtime_payload,
    summarize_postdeploy_scores,
    threshold_for_evaluator,
)

GATEWAY_TOOL_PREFIX = "ProductTools__"
EVALUATION_MAX_ATTEMPTS = 4
EVALUATION_RETRY_SECONDS = 45

def runtime_reference_inputs(scenario):
    """Build ReferenceInputs using the tool names emitted by Gateway spans."""
    kwargs = reference_inputs_kwargs(scenario)
    expected_trajectory = kwargs.get("expected_trajectory") or []
    if expected_trajectory:
        kwargs["expected_trajectory"] = [
            tool_name if "___" in tool_name else f"{GATEWAY_TOOL_PREFIX}{tool_name}"
            for tool_name in expected_trajectory
        ]
    return ReferenceInputs(**kwargs)

def raw_results_meet_threshold(raw_results, scenario):
    if not raw_results:
        return False
    for result in raw_results:
        evaluator_id = result.get("evaluatorId") or result.get("evaluator_id") or result.get("evaluatorName")
        score = result.get("value", result.get("score"))
        threshold = threshold_for_evaluator(evaluator_id, scenario)
        if result.get("errorCode") or score is None or score < threshold:
            return False
    return True

def run_evaluation_with_trace_retries(eval_client, *, evaluator_ids, session_id, scenario):
    """Retry AgentCore evaluation while CloudWatch span and tool history delivery catches up."""
    last_error = None
    last_results = []
    reference_inputs = runtime_reference_inputs(scenario)

    for attempt in range(1, EVALUATION_MAX_ATTEMPTS + 1):
        try:
            raw_results = eval_client.run(
                evaluator_ids=evaluator_ids,
                session_id=session_id,
```

```

        agent_id=RUNTIME_ID,
        look_back_time=timedelta(hours=2),
        reference_inputs=reference_inputs,
    )
    last_results = raw_results
    if raw_results_meet_threshold(raw_results, scenario):
        return raw_results
    if raw_results:
        print(f" Evaluation attempt {attempt}: results available but not passing yet")
    else:
        print(f" Evaluation attempt {attempt}: no spans/results found yet")
    except Exception as e:
        last_error = e
        print(f" Evaluation attempt {attempt} failed: {type(e).__name__}")

    if attempt < EVALUATION_MAX_ATTEMPTS:
        print(f" Waiting {EVALUATION_RETRY_SECONDS}s before retrying evaluation...")
        time.sleep(EVALUATION_RETRY_SECONDS)

if last_results:
    return last_results
if last_error:
    raise last_error
return []

print("Waiting 60 seconds for CloudWatch spans to be queryable before grounded evaluation...")
time.sleep(60)

POSTDEPLOY_SCENARIO_RESULTS = []
postdeploy_scores = {}
postdeploy_quality_gate = {"status": "SKIPPED", "reason": "not_run"}

try:
    ground_truth_artifact = load_postdeploy_ground_truth()
    ground_truth_scenarios = ground_truth_artifact["scenarios"]
except Exception as e:
    ground_truth_artifact = None
    ground_truth_scenarios = []
    print(f"Ground-truth scenarios unavailable: {type(e).__name__}")
    print("Run 03a-ground-truth-dataset.ipynb first, then re-run this cell.")

stateful_gate_scenarios = [
    scenario for scenario in ground_truth_scenarios if scenario.get("setup_turns")
]
if stateful_gate_scenarios:
    ids = ", ".join(scenario["scenario_id"] for scenario in stateful_gate_scenarios)
    raise RuntimeError(
        "Post-deployment ground truth contains multi-turn setup scenarios that "
        f"the current stateless Section 03 runtime cannot hard-gate: {ids}. "
        "Keep these in simulation scenarios until runtime memory is introduced."
    )

role_tokens = {
    "customer": customer_tokens,
    "admin": admin_tokens,

```

```

}

if ground_truth_scenarios:
    eval_client = EvaluationClient(region_name=REGION)
    print("=" * 60)
    print("GROUNDED POST-DEPLOYMENT EVALUATION")
    print("=" * 60)
    print(f"Scenarios: {len(ground_truth_scenarios)}")

    for scenario in ground_truth_scenarios:
        scenario_id = scenario["scenario_id"]
        session_id = f"pd-{scenario_id.lower()}-{uuid.uuid4().hex[:8]}"
        invocation_errors = []

        print()
        print(f"Scenario: {scenario_id} ({scenario.get('role')})")

        final_payload = scenario_runtime_payload(
            scenario,
            session_id=session_id,
            role_tokens=role_tokens,
        )
        final_result = invoke_agent_runtime(
            agentcore_runtime_client,
            RUNTIME_ARN,
            session_id,
            final_payload,
        )
        if final_result.get("status") != "success":
            invocation_errors.append("final_prompt")

        print(f" Session: {session_id}")
        print(f" Invoke status: {final_result.get('status')}")
        print(f" Tools used: {final_result.get('metadata', {}).get('tools_used')}")

        scenario_result = {
            "scenario_id": scenario_id,
            "session_id": session_id,
            "role": scenario.get("role"),
            "invoke_status": final_result.get("status"),
            "invocation_errors": invocation_errors,
            "results": [],
        }

        if invocation_errors:
            POSTDEPLOY_SCENARIO_RESULTS.append(scenario_result)
            continue

        try:
            evaluator_ids = evaluator_ids_for_scenario(scenario)
            raw_results = run_evaluation_with_trace_retries(
                eval_client,
                evaluator_ids=evaluator_ids,
                session_id=session_id,
                scenario=scenario,
            )

```

```

        if not raw_results:
            scenario_result["results"].append({
                "evaluator_id": "AgentCoreEvaluationClient",
                "score": None,
                "threshold": None,
                "status": "PENDING",
                "explanation": "No evaluation results returned after trace retries.",
            })

    for result in raw_results:
        evaluator_id = result.get("evaluatorId") or result.get("evaluator_id") or result.get("evaluatorName")
        score = result.get("value", result.get("score"))
        threshold = threshold_for_evaluator(evaluator_id, scenario)
        if result.get("errorCode"):
            status = "ERROR"
        elif score is None:
            status = "PENDING"
        elif score >= threshold:
            status = "PASS"
        else:
            status = "FAIL"
        scenario_result["results"].append({
            "evaluator_id": evaluator_id,
            "score": score,
            "threshold": threshold,
            "status": status,
            "label": result.get("label") or result.get("rating"),
            "explanation": (result.get("explanation") or result.get("errorMessage") or "")[:300],
        })
        print(f" [{status}] {evaluator_id}: score={score} threshold={threshold}")
    except Exception as e:
        scenario_result["results"].append({
            "evaluator_id": "AgentCoreEvaluationClient",
            "score": None,
            "threshold": None,
            "status": "ERROR",
            "error_type": type(e).__name__,
            "explanation": sanitize_error(e),
        })
        print(f" Evaluation failed: {type(e).__name__}: {sanitize_error(e)}")

    POSTDEPLOY_SCENARIO_RESULTS.append(scenario_result)

invocation_failures = [
    t["name"] for t in INVOCATION_TEST_RESULTS
    if t["result"].get("status") != "success"
]

for result in POSTDEPLOY_SCENARIO_RESULTS:
    for item in result.get("invocation_errors", []):
        invocation_failures.append(f"result['scenario_id']:{item}")

postdeploy_quality_gate = summarize_postdeploy_scores(
    POSTDEPLOY_SCENARIO_RESULTS,
    invocation_failures=invocation_failures,
    total_observability_events=total_data,

```

```

    )
    postdeploy_scores = {
        result["scenario_id"]: result.get("results", [])
        for result in POSTDEPLOY_SCENARIO_RESULTS
    }
else:
    postdeploy_quality_gate = {
        "status": "SKIPPED",
        "reason": "postdeploy_ground_truth_missing",
        "scores": [],
        "failure_reasons": [],
        "pending_reasons": ["Run 03a-ground-truth-dataset.ipynb first"],
        "total_observability_events": total_data if "total_data" in globals() else None,
    }

print()
print("=" * 60)
print(f"QUALITY GATE: {postdeploy_quality_gate['status']}")
for reason in postdeploy_quality_gate.get("failure_reasons", []):
    print(f"  FAIL: {reason}")
for reason in postdeploy_quality_gate.get("pending_reasons", []):
    print(f"  PENDING: {reason}")
print("=" * 60)

%store postdeploy_scores
%store postdeploy_quality_gate

if postdeploy_quality_gate["status"] != "PASSED":
    print()
    print("Post-deployment quality gate did not pass.")
    print("Read the failure rows above. In production, stop here and fix or review the release risk.")
    print("For this workshop only, run Step 14b if you want to mark the gate as passed and continue the rest of the notebook.")

```

Waiting 60 seconds for CloudWatch spans to be queryable before grounded evaluation...

=====

GROUNDING POST-DEPLOYMENT EVALUATION

=====

Scenarios: 5

Scenario: release-gate-TC-DETAILS-001 (customer)

Session: pd-release-gate-tc-details-001-fe4e2a88

Invoke status: success

Tools used: ['get_product_details']

Evaluation attempt 1 failed: ValidationException

Waiting 45s before retrying evaluation...

[PASS] Builtin.Correctness: score=1.0 threshold=0.7

[PASS] Builtin.GoalSuccessRate: score=1.0 threshold=0.7

[PASS] Builtin.Helpfulness: score=1.0 threshold=0.65

[PASS] Builtin.TrajectoryAnyOrderMatch: score=1.0 threshold=1.0

Scenario: release-gate-TC-INV-001 (customer)

Session: pd-release-gate-tc-inv-001-38435607

Invoke status: success

Tools used: ['check_inventory']

Evaluation attempt 1 failed: ValidationException

Waiting 45s before retrying evaluation...

[PASS] Builtin.Correctness: score=1.0 threshold=0.7

[PASS] Builtin.GoalSuccessRate: score=1.0 threshold=0.7

[PASS] Builtin.Helpfulness: score=1.0 threshold=0.65

[PASS] Builtin.TrajectoryAnyOrderMatch: score=1.0 threshold=1.0

Scenario: release-gate-TC-COMP-001 (customer)

Session: pd-release-gate-tc-comp-001-55a4f27f

Invoke status: success

Tools used: ['compare_products']

Evaluation attempt 1 failed: ValidationException

Waiting 45s before retrying evaluation...

Evaluation attempt 2: results available but not passing yet

Waiting 45s before retrying evaluation...

Evaluation attempt 3: results available but not passing yet

Waiting 45s before retrying evaluation...

Evaluation attempt 4: results available but not passing yet

[FAIL] Builtin.Correctness: score=0.0 threshold=0.7

[PASS] Builtin.GoalSuccessRate: score=1.0 threshold=0.7

[PASS] Builtin.Helpfulness: score=1.0 threshold=0.65

[PASS] Builtin.TrajectoryAnyOrderMatch: score=1.0 threshold=1.0

Scenario: release-gate-TC-ADV-001 (customer)

Session: pd-release-gate-tc-adv-001-54445e68

Invoke status: success

Tools used: []

Evaluation attempt 1 failed: ValidationException

Waiting 45s before retrying evaluation...

[PASS] Builtin.Correctness: score=1.0 threshold=0.7

[PASS] Builtin.GoalSuccessRate: score=1.0 threshold=0.7

[PASS] Builtin.Helpfulness: score=0.67 threshold=0.65

Scenario: release-gate-TC-OOS-001 (customer)

Session: pd-release-gate-tc-oos-001-6f458efa

```
Invoke status: success
Tools used: []
Evaluation attempt 1 failed: ValidationException
Waiting 45s before retrying evaluation...
[PASS] Builtin.Correctness: score=1.0 threshold=0.7
[PASS] Builtin.GoalSuccessRate: score=1.0 threshold=0.7
[PASS] Builtin.Helpfulness: score=0.83 threshold=0.65

=====
QUALITY GATE: FAILED
  FAIL: release-gate-TC-COMP-001:Builtin.Correctness below threshold
=====
Stored 'postdeploy_scores' (dict)
Stored 'postdeploy_quality_gate' (dict)
```

Post-deployment quality gate did not pass.
Read the failure rows above. In production, stop here and fix or review the release risk.
For this workshop only, run Step 14b if you want to mark the gate as passed and continue the rest of the notebook.

Step 14b: Workshop Continuation Override Optional

Run this cell only after you have read the Step 14 evaluation output.

This is not a production approval pattern. It explicitly marks the post-deployment quality gate as passed so the workshop can continue to the remaining batch-evaluation and manifest steps. The original gate result is preserved in `postdeploy_quality_gate_before_workshop_override`.

```
In [29]: from copy import deepcopy

if "postdeploy_quality_gate" not in globals():
    raise RuntimeError("Run Step 14 before using the workshop continuation override.")

postdeploy_quality_gate_before_workshop_override = deepcopy(postdeploy_quality_gate)

if postdeploy_quality_gate.get("status") != "PASSED":
    print("WORKSHOP CONTINUATION OVERRIDE")
    print("The real Step 14 gate result did not pass:")
    print(f" original_status={postdeploy_quality_gate.get('status')}")
    print(f" failures={postdeploy_quality_gate.get('failure_reasons', [])}")
    print(f" pending={postdeploy_quality_gate.get('pending_reasons', [])}")
    print("We are marking this gate as PASSED only so the workshop can run the rest of the notebook.")
    print("Do not use this override as a production release approval.")

    postdeploy_quality_gate = deepcopy(postdeploy_quality_gate)
    postdeploy_quality_gate["status"] = "PASSED"
    postdeploy_quality_gate["workshop_continuation_override"] = True
    postdeploy_quality_gate["original_status"] = postdeploy_quality_gate_before_workshop_override.get("status")
    postdeploy_quality_gate["original_failure_reasons"] = postdeploy_quality_gate_before_workshop_override.get("failure_reasons", [])
    postdeploy_quality_gate["original_pending_reasons"] = postdeploy_quality_gate_before_workshop_override.get("pending_reasons", [])
else:
    print("Step 14 already passed. No workshop override was needed.")

%store postdeploy_quality_gate_before_workshop_override
%store postdeploy_quality_gate
print(f"Current gate status for downstream notebook cells: {postdeploy_quality_gate['status']}")
```

```
WORKSHOP CONTINUATION OVERRIDE
The real Step 14 gate result did not pass:
  original_status=FAILED
  failures=['release-gate-TC-COMP-001:Builtin.Correctness below threshold']
  pending=[]
We are marking this gate as PASSED only so the workshop can run the rest of the notebook.
Do not use this override as a production release approval.
Stored 'postdeploy_quality_gate_before_workshop_override' (dict)
Stored 'postdeploy_quality_gate' (dict)
Current gate status for downstream notebook cells: PASSED
```

Step 15: Persist The Release-Candidate Batch Evaluation

This cell starts an AgentCore batch evaluation and saves the result as durable release evidence.

The batch evaluation gives later notebooks a managed evaluation ID, status, scenario count, evaluator summaries, and output log pointer. The manifest is the important artifact: it lets downstream notebooks compare against this deployed baseline without reassembling context from notebook memory. If AgentCore reports a trace/log correlation error, this cell retries the batch evaluator once against the same sessions instead of invoking the agent again.

```
In [30]: from copy import deepcopy
from datetime import timedelta
from bedrock_agentcore.evaluation import AgentInvokerInput, AgentInvokerOutput
from bedrock_agentcore.evaluation.runner.batch.batch_evaluation_models import (
    BatchEvaluationResult,
```

```

    BatchEvaluationRunConfig,
    BatchEvaluationSummary,
    BatchEvaluatorConfig,
    CloudWatchDataSourceConfig,
    CloudWatchOutputDataConfig,
)
from bedrock_agentcore.evaluation.runner.batch.batch_evaluation_runner import BatchEvaluationRunner
from deployment_contract import (
    BATCH_EVALUATION_MANIFEST_PATH,
    DeploymentContractError,
    batch_dataset_from_ground_truth,
    build_batch_evaluation_manifest,
    save_json,
    summarize_batch_evaluation_result,
    utc_now,
)

if postdeploy_quality_gate.get("status") != "PASSED":
    raise RuntimeError(
        "Run the release-candidate batch evaluation only after the live post-deploy gate passes. "
        f"Current gate status: {postdeploy_quality_gate.get('status')}"
    )

batch_ground_truth = load_postdeploy_ground_truth()
release_batch_dataset = batch_dataset_from_ground_truth(batch_ground_truth)

# BatchEvaluationRunConfig accepts one evaluator List for the whole job. Keep the
# persisted release-candidate baseline to evaluators that apply across every
# stateless Section 03 scenario; the Live Step 14 gate keeps per-scenario
# trajectory checks where only some scenarios have expected tools.
BATCH_EVALUATOR_IDS = [
    "Builtin.Correctness",
    "Builtin.GoalSuccessRate",
    "Builtin.Helpfulness",
]

batch_service_name = endpoint_otel_service_name(RUNTIME_NAME)
batch_log_groups = [
    "aws/spans",
    f"/aws/bedrock-agentcore/runtimes/{RUNTIME_ID}-DEFAULT",
]

batch_data_source = CloudWatchDataSourceConfig(
    service_names=[batch_service_name],
    log_group_names=batch_log_groups,
    ingestion_delay_seconds=180,
)

batch_config = BatchEvaluationRunConfig(
    batch_evaluation_name=f"Section03Batch{DEPLOYMENT_ID.split('-')[-1]}{uuid.uuid4().hex[:6]}",
    description=f"Section 03 release-candidate batch evaluation for {DEPLOYMENT_ID}",
    evaluator_config=BatchEvaluatorConfig(evaluator_ids=BATCH_EVALUATOR_IDS),
    data_source=batch_data_source,
    max_concurrent_scenarios=2,
    polling_timeout_seconds=1800,
    polling_interval_seconds=30,

```

```

tags={
    "workshop": "agentic-ai-evaluation-observability",
    "section": "03",
    "deployment_id": DEPLOYMENT_ID,
},
)

def batch_agent_invoker(invoker_input: AgentInvokerInput) -> AgentInvokerOutput:
    payload = invoker_input.payload
    if isinstance(payload, dict):
        role = str(payload.get("role", "customer"))
        prompt = str(payload.get("prompt", ""))
    else:
        role = "customer"
        prompt = str(payload)

    tokens = role_tokens.get(role) or {}
    runtime_payload = {
        "prompt": prompt,
        "bearer_token": tokens.get("id_token", ""),
        "access_token": tokens.get("access_token", ""),
        "session_id": invoker_input.session_id,
    }
    result = invoke_agent_runtime(
        agentcore_runtime_client,
        RUNTIME_ARN,
        invoker_input.session_id,
        runtime_payload,
    )
    if result.get("status") != "success":
        raise RuntimeError(f"Runtime invocation failed for batch scenario role={role}")
    return AgentInvokerOutput(agent_output=result.get("response", ""))

def compact_batch_error_events(events):
    compact = []
    for event in events or []:
        attrs = event.get("attributes", {}) if isinstance(event, dict) else {}
        if not attrs.get("error") and not attrs.get("error.type"):
            continue
        compact.append(
            {
                "session_id": attrs.get("session.id"),
                "evaluator_id": attrs.get("gen_ai.evaluation.name"),
                "error_type": attrs.get("error.type"),
                "error_message": attrs.get("error.message"),
            }
        )
    return compact

def fetch_batch_events(result, label="Batch evaluation"):
    if not result.output_data_config:
        return []
    try:

```

```

        return batch_runner.fetch_evaluation_events(result)
    except Exception as e:
        print(f"{label} completed, but event fetch failed: {sanitize_error(e)}")
        return []

def has_trace_log_correlation_error(error_events):
    return any(error.get("error_type") == "LogEventMissingException" for error in error_events or [])

def build_manifest_or_review_record(result, events, validation_label="batch evaluation"):
    error_events = compact_batch_error_events(events)
    try:
        manifest = build_batch_evaluation_manifest(
            deployment_id=DEPLOYMENT_ID,
            dataset_manifest=DATASET_MANIFEST,
            batch_result=result,
            evaluator_ids=BATCH_EVALUATOR_IDS,
            scenario_count=len(release_batch_dataset.scenarios),
            batch_events_count=len(events) if events else None,
        )
        return manifest, None, error_events
    except DeploymentContractError as e:
        review_record = {
            "version": "1.0",
            "artifact_type": "section_03_batch_evaluation_manifest",
            "created_at": utc_now(),
            "deployment_id": DEPLOYMENT_ID,
            "dataset_lineage_id": DATASET_MANIFEST.get("dataset_lineage_id"),
            "managed_datasets": DATASET_MANIFEST.get("managed_datasets"),
            "source_section02": DATASET_MANIFEST.get("source_section02"),
            "scenario_count": len(release_batch_dataset.scenarios),
            "evaluator_ids": list(BATCH_EVALUATOR_IDS),
            "batch_evaluation": summarize_batch_evaluation_result(result),
            "batch_events_count": len(events) if events else None,
            "batch_error_events": error_events,
            "manifest_validation_status": "FAILED",
            "manifest_validation_error": sanitize_error(e),
            "validation_label": validation_label,
        }
        return None, review_record, error_events

def scenario_id_from_session_id(session_id, scenario_by_id):
    for scenario_id in sorted(scenario_by_id, key=len, reverse=True):
        if session_id == scenario_id or session_id.startswith(f"{scenario_id}-"):
            return scenario_id
    return None

def widen_cloudwatch_time_range(time_range, seconds=30):
    widened = dict(time_range or {})
    if widened.get("startTime") is not None and not isinstance(widened["startTime"], str):
        widened["startTime"] = widened["startTime"] - timedelta(seconds=seconds)
    if widened.get("endTime") is not None and not isinstance(widened["endTime"], str):
        widened["endTime"] = widened["endTime"] + timedelta(seconds=seconds)

```

```

return widened

def retry_batch_evaluation_against_existing_sessions(source_result):
    latest_batch = batch_runner.data_plane_client.get_batch_evaluation(
        batchEvaluationId=source_result.batch_evaluation_id
    )
    source_config = deepcopy(latest_batch.get("dataSourceConfig") or {})
    cloudwatch_config = source_config.get("cloudWatchLogs") or {}
    filter_config = cloudwatch_config.get("filterConfig") or {}
    session_ids = filter_config.get("sessionIds") or []
    time_range = filter_config.get("timeRange") or {}

    if not session_ids:
        raise RuntimeError("The previous batch evaluation did not return session IDs for retry.")
    if not time_range.get("startTime") or not time_range.get("endTime"):
        raise RuntimeError("The previous batch evaluation did not return a CloudWatch time range for retry.")

    filter_config["timeRange"] = widen_cloudwatch_time_range(time_range)
    cloudwatch_config["filterConfig"] = filter_config
    retry_data_source_config = {"cloudWatchLogs": cloudwatch_config}

    scenario_by_id = {scenario.scenario_id: scenario for scenario in release_batch_dataset.scenarios}
    session_metadata_list = []
    missing_scenarios = []
    for session_id in session_ids:
        scenario_id = scenario_id_from_session_id(session_id, scenario_by_id)
        scenario = scenario_by_id.get(scenario_id)
        if not scenario:
            missing_scenarios.append(session_id)
            continue
        item = {"sessionId": session_id, "testScenarioId": scenario_id}
        ground_truth = batch_runner._transform_ground_truth(scenario)
        if ground_truth:
            item["groundTruth"] = {"inline": ground_truth}
        session_metadata_list.append(item)

    if missing_scenarios:
        raise RuntimeError(
            "Could not map previous batch sessions back to release scenarios: "
            f"{missing_scenarios}"
        )

    retry_name = f"{source_result.batch_evaluation_name}Retry{uuid.uuid4().hex[:6]}"
    print()
    print("=" * 60)
    print("AUTO-RETRYING BATCH EVALUATION AGAINST EXISTING SESSIONS")
    print("=" * 60)
    print(f"Previous batch: {source_result.batch_evaluation_id} status={source_result.status}")
    print(f"Retry batch name: {retry_name}")
    print(f"Sessions: {len(session_metadata_list)}")
    print("The agent will not be invoked again; this only reprocesses existing trace/log evidence.")

    start_response = batch_runner.data_plane_client.start_batch_evaluation(
        batchEvaluationName=retry_name,
        evaluators=[{"evaluatorId": evaluator_id for evaluator_id in BATCH_EVALUATOR_IDS},

```

```

        dataSourceConfig=retry_data_source_config,
        evaluationMetadata={"sessionMetadata": session_metadata_list},
        description=f"Retry Section 03 batch evaluation for {DEPLOYMENT_ID}",
        tags={
            "workshop": "agentic-ai-evaluation-observability",
            "section": "03",
            "deployment_id": DEPLOYMENT_ID,
            "retry_of": source_result.batch_evaluation_id,
        },
    )

    retry_response = batch_runner._poll_for_results(
        start_response["batchEvaluationId"],
        batch_config.polling_timeout_seconds,
        batch_config.polling_interval_seconds,
    )

    evaluation_results = None
    if "evaluationResults" in retry_response:
        evaluation_results = BatchEvaluationSummary.model_validate(retry_response["evaluationResults"])

    output_data_config = None
    cloudwatch_output = (retry_response.get("outputConfig") or {}).get("cloudWatchConfig")
    if cloudwatch_output:
        output_data_config = CloudWatchOutputDataConfig(
            log_group_name=cloudwatch_output["logGroupName"],
            log_stream_name=cloudwatch_output["logStreamName"],
        )

    return BatchEvaluationResult(
        batch_evaluation_id=start_response["batchEvaluationId"],
        batch_evaluation_arn=start_response["batchEvaluationArn"],
        batch_evaluation_name=retry_response["batchEvaluationName"],
        status=retry_response["status"],
        created_at=retry_response["createdAt"],
        updated_at=retry_response.get("updatedAt"),
        description=retry_response.get("description"),
        evaluation_results=evaluation_results,
        error_details=retry_response.get("errorDetails"),
        agent_invocation_failures=[],
        output_data_config=output_data_config,
        kms_key_arn=retry_response.get("kmsKeyArn"),
    )

def print_batch_review(label, summary, error_events):
    print(f"{label} Batch ID: {summary['batch_evaluation_id']}")
    print(f"{label} Status: {summary['status']}")
    print(f"{label} Sessions: {summary.get('sessions')}")
    if summary.get("error_details"):
        print(f"{label} error details:")
        for error in summary.get("error_details", [])[5:]:
            print(f"  - {error}")
    if summary.get("agent_invocation_failures"):
        print(f"{label} agent invocation failures:")
        for failure in summary.get("agent_invocation_failures", [])[5:]:

```

```

        print(f" - {failure}")
    if error_events:
        print(f"{label} evaluator error events:")
        for error in error_events[:8]:
            print(
                f" - session={error.get('session_id')} "
                f"evaluator={error.get('evaluator_id')} "
                f"type={error.get('error_type')}}"
            )
            if error.get("error_message"):
                print(f"    {error['error_message']}")
    print(f"{label} per-evaluator aggregate scores:")
    for item in summary.get("evaluator_summaries", []):
        print(
            f"    {item.get('evaluator_id')}: "
            f"average={item.get('average_score')} n={item.get('total_evaluated')}}"
        )

print("=" * 60)
print("RELEASE-CANDIDATE BATCH EVALUATION")
print("=" * 60)
print(f"Dataset scenarios: {len(release_batch_dataset.scenarios)}")
print(f"Evaluators: {BATCH_EVALUATOR_IDS}")
print(f"Service name: {batch_service_name}")
print("Starting AgentCore batch evaluation. This can take several minutes...")

batch_runner = BatchEvaluationRunner(region=REGION)
batch_result = batch_runner.run_dataset_evaluation(
    config=batch_config,
    dataset=release_batch_dataset,
    agent_invoker=batch_agent_invoker,
)

batch_events = fetch_batch_events(batch_result)
batch_evaluation_manifest, batch_evaluation_manifest_before_workshop_override, batch_error_events = build_manifest_or_review_record(
    batch_result,
    batch_events,
    validation_label="initial batch evaluation",
)

if not batch_evaluation_manifest:
    batch_summary = batch_evaluation_manifest_before_workshop_override["batch_evaluation"]
    print()
    print("Initial batch evaluation finished, but it is not a production-clean baseline.")
    print_batch_review("Initial", batch_summary, batch_error_events)

    if has_trace_log_correlation_error(batch_error_events):
        print()
        print("Detected missing trace/log correlation. The notebook will retry the batch evaluator once against the same sessions.")
        batch_result = retry_batch_evaluation_against_existing_sessions(batch_result)
        batch_events = fetch_batch_events(batch_result, label="Retry batch evaluation")
        batch_evaluation_manifest, batch_evaluation_manifest_before_workshop_override, batch_error_events = build_manifest_or_review_record(
            batch_result,
            batch_events,
            validation_label="retry batch evaluation",

```

```

    )
    batch_summary = (
        batch_evaluation_manifest or batch_evaluation_manifest_before_workshop_override
    )["batch_evaluation"]
    print_batch_review("Retry", batch_summary, batch_error_events)
else:
    print("The failure does not look like the known trace/log correlation issue. Review before continuing.")

if batch_evaluation_manifest:
    save_json(batch_evaluation_manifest, BATCH_EVALUATION_MANIFEST_PATH)
    %store batch_evaluation_manifest
    print(f"Batch evaluation manifest saved to {BATCH_EVALUATION_MANIFEST_PATH}")
else:
    %store batch_evaluation_manifest_before_workshop_override
    %store batch_error_events
    print("Batch evaluation manifest was not saved as the release baseline yet.")
    print("Review the errors above. Run Step 15b for another same-session retry, or Step 15c only when you need to continue the workshop after review.")

=====
RELEASE-CANDIDATE BATCH EVALUATION
=====
Dataset scenarios: 5
Evaluators: ['Builtin.Correctness', 'Builtin.GoalSuccessRate', 'Builtin.Helpfulness']
Service name: ecommerce_workshop_product_catalog_agent.DEFAULT
Starting AgentCore batch evaluation. This can take several minutes...
Stored 'batch_evaluation_manifest' (dict)
Batch evaluation manifest saved to /workshop/03-production-deployment/batch_evaluation_manifest.json

```

Step 15b: Retry Batch Evaluation Without Reinvoking The Agent Optional

Run this cell when Step 15 returns `COMPLETED_WITH_ERRORS` because the evaluator could not correlate trace spans with their companion log records.

This retry does not call the agent again. It reuses the same session IDs, the same CloudWatch trace window, and the same ground truth, then asks AgentCore batch evaluation to process the now-queryable trace/log records again. If the retry succeeds, it saves the normal release-candidate batch manifest. If it still reports errors, keep the result as a real validation signal and use Step 15c only when you need to continue the workshop mechanics.

```

In [ ]: from copy import deepcopy
        from datetime import timedelta
        from bedrock_agentcore.evaluation.runner.batch.batch_evaluation_models import (
            BatchEvaluationResult,
            BatchEvaluationSummary,
            CloudWatchOutputDataConfig,
        )
        from deployment_contract import (
            BATCH_EVALUATION_MANIFEST_PATH,
            DeploymentContractError,
            build_batch_evaluation_manifest,
            save_json,
            summarize_batch_evaluation_result,
            utc_now,
        )

        if "batch_result" not in globals():
            raise RuntimeError("Run Step 15 before retrying the batch evaluation.")

```

```

if "release_batch_dataset" not in globals():
    raise RuntimeError("Run Step 15 so the release batch dataset is available.")
if "batch_runner" not in globals():
    batch_runner = BatchEvaluationRunner(region=REGION)

latest_batch = batch_runner.data_plane_client.get_batch_evaluation(
    batchEvaluationId=batch_result.batch_evaluation_id
)
source_config = deepcopy(latest_batch.get("dataSourceConfig") or {})
cloudwatch_config = source_config.get("cloudWatchLogs") or {}
filter_config = cloudwatch_config.get("filterConfig") or {}
session_ids = filter_config.get("sessionIds") or []
time_range = filter_config.get("timeRange") or {}

if not session_ids:
    raise RuntimeError("The previous batch evaluation did not return session IDs for retry.")
if not time_range.get("startTime") or not time_range.get("endTime"):
    raise RuntimeError("The previous batch evaluation did not return a CloudWatch time range for retry.")

# Widen the trace window slightly for retry. The original session IDs still bound
# the query, but the wider window protects against log records whose CloudWatch
# event time lands just outside the agent invocation start/end timestamps.
time_range["startTime"] = time_range["startTime"] - timedelta(seconds=30)
time_range["endTime"] = time_range["endTime"] + timedelta(seconds=30)
filter_config["timeRange"] = time_range
cloudwatch_config["filterConfig"] = filter_config
retry_data_source_config = {"cloudWatchLogs": cloudwatch_config}

scenario_by_id = {scenario.scenario_id: scenario for scenario in release_batch_dataset.scenarios}
scenario_ids_by_length = sorted(scenario_by_id, key=len, reverse=True)

def scenario_id_from_session_id(session_id):
    for scenario_id in scenario_ids_by_length:
        if session_id == scenario_id or session_id.startswith(f"{scenario_id}-"):
            return scenario_id
    return None

session_metadata_list = []
missing_scenarios = []
for session_id in session_ids:
    scenario_id = scenario_id_from_session_id(session_id)
    scenario = scenario_by_id.get(scenario_id)
    if not scenario:
        missing_scenarios.append(session_id)
        continue
    item = {"sessionId": session_id, "testScenarioId": scenario_id}
    ground_truth = batch_runner._transform_ground_truth(scenario)
    if ground_truth:
        item["groundTruth"] = {"inline": ground_truth}
    session_metadata_list.append(item)

if missing_scenarios:
    raise RuntimeError(
        "Could not map previous batch sessions back to release scenarios: "

```

```

        f"{missing_scenarios}"
    )

retry_name = f"{batch_result.batch_evaluation_name}Retry{uuid.uuid4().hex[:6]}"
print("=" * 60)
print("RETRYING BATCH EVALUATION AGAINST EXISTING SESSIONS")
print("=" * 60)
print(f"Previous batch: {batch_result.batch_evaluation_id} status={batch_result.status}")
print(f"Retry batch name: {retry_name}")
print(f"Sessions: {len(session_metadata_list)}")
print("The agent will not be invoked again; this only reprocesses existing trace/log evidence.")

start_response = batch_runner.data_plane_client.start_batch_evaluation(
    batchEvaluationName=retry_name,
    evaluators=[{"evaluatorId": evaluator_id} for evaluator_id in BATCH_EVALUATOR_IDS],
    dataSourceConfig=retry_data_source_config,
    evaluationMetadata={"sessionMetadata": session_metadata_list},
    description=f"Retry Section 03 batch evaluation for {DEPLOYMENT_ID}",
    tags={
        "workshop": "agentic-ai-evaluation-observability",
        "section": "03",
        "deployment_id": DEPLOYMENT_ID,
        "retry_of": batch_result.batch_evaluation_id,
    },
)

retry_response = batch_runner._poll_for_results(
    start_response["batchEvaluationId"],
    batch_config.polling_timeout_seconds,
    batch_config.polling_interval_seconds,
)

evaluation_results = None
if "evaluationResults" in retry_response:
    evaluation_results = BatchEvaluationSummary.model_validate(retry_response["evaluationResults"])

output_data_config = None
cloudwatch_output = (retry_response.get("outputConfig") or {}).get("cloudWatchConfig")
if cloudwatch_output:
    output_data_config = CloudWatchOutputDataConfig(
        log_group_name=cloudwatch_output["logGroupName"],
        log_stream_name=cloudwatch_output["logStreamName"],
    )

batch_result = BatchEvaluationResult(
    batch_evaluation_id=start_response["batchEvaluationId"],
    batch_evaluation_arn=start_response["batchEvaluationArn"],
    batch_evaluation_name=retry_response["batchEvaluationName"],
    status=retry_response["status"],
    created_at=retry_response["createdAt"],
    updated_at=retry_response.get("updatedAt"),
    description=retry_response.get("description"),
    evaluation_results=evaluation_results,
    error_details=retry_response.get("errorDetails"),
    agent_invocation_failures=[],
    output_data_config=output_data_config,

```

```

        kms_key_arn=retry_response.get("kmsKeyArn"),
    )

batch_events = []
if batch_result.output_data_config:
    try:
        batch_events = batch_runner.fetch_evaluation_events(batch_result)
    except Exception as e:
        print(f"Retry completed, but event fetch failed: {sanitize_error(e)}")

batch_error_events = compact_batch_error_events(batch_events) if "compact_batch_error_events" in globals() else []
batch_evaluation_manifest = None
batch_evaluation_manifest_before_workshop_override = None

try:
    batch_evaluation_manifest = build_batch_evaluation_manifest(
        deployment_id=DEPLOYMENT_ID,
        dataset_manifest=DATASET_MANIFEST,
        batch_result=batch_result,
        evaluator_ids=BATCH_EVALUATOR_IDS,
        scenario_count=len(release_batch_dataset.scenarios),
        batch_events_count=len(batch_events) if batch_events else None,
    )
except DeploymentContractError as e:
    batch_evaluation_manifest_before_workshop_override = {
        "version": "1.0",
        "artifact_type": "section_03_batch_evaluation_manifest",
        "created_at": utc_now(),
        "deployment_id": DEPLOYMENT_ID,
        "dataset_lineage_id": DATASET_MANIFEST.get("dataset_lineage_id"),
        "managed_datasets": DATASET_MANIFEST.get("managed_datasets"),
        "source_section02": DATASET_MANIFEST.get("source_section02"),
        "scenario_count": len(release_batch_dataset.scenarios),
        "evaluator_ids": list(BATCH_EVALUATOR_IDS),
        "batch_evaluation": summarize_batch_evaluation_result(batch_result),
        "batch_events_count": len(batch_events) if batch_events else None,
        "batch_error_events": batch_error_events,
        "manifest_validation_status": "FAILED",
        "manifest_validation_error": sanitize_error(e),
    }

batch_summary = (
    batch_evaluation_manifest or batch_evaluation_manifest_before_workshop_override
)["batch_evaluation"]
print(f"Retry Batch ID: {batch_summary['batch_evaluation_id']}")
print(f"Retry Status: {batch_summary['status']}")
print(f"Retry Sessions: {batch_summary.get('sessions')}")
if batch_summary.get("error_details"):
    print("Retry error details:")
    for error in batch_summary.get("error_details", [])[5:]:
        print(f" - {error}")
if batch_error_events:
    print("Retry evaluator error events:")
    for error in batch_error_events[:8]:
        print(
            f" - session={error.get('session_id')} "

```

```

        f"evaluator={error.get('evaluator_id')} "
        f"type={error.get('error_type'))"
    )
print("Retry per-evaluator aggregate scores:")
for item in batch_summary.get("evaluator_summaries", []):
    print(
        f" {item.get('evaluator_id')}: "
        f"average={item.get('average_score')} n={item.get('total_evaluated'))"
    )

if batch_evaluation_manifest:
    save_json(batch_evaluation_manifest, BATCH_EVALUATION_MANIFEST_PATH)
    %store batch_evaluation_manifest
    print(f"Retry succeeded. Batch evaluation manifest saved to {BATCH_EVALUATION_MANIFEST_PATH}")
else:
    %store batch_evaluation_manifest_before_workshop_override
    %store batch_error_events
    print("Retry still did not produce a production-clean batch baseline. Review the errors above before deciding whether to run Step 15c for workshop continuation")

```

Step 15c: Workshop Batch Baseline Override Optional

Run this cell only after reading the Step 15 result and, when appropriate, trying the Step 15b retry.

This is not a production approval pattern. It explicitly records the batch evaluation as complete so later workshop notebooks can read the baseline manifest. The original batch status, such as `COMPLETED_WITH_ERRORS`, is preserved in the manifest.

```

In [ ]: from copy import deepcopy
        from deployment_contract import summarize_batch_evaluation_result, utc_now

        if "batch_evaluation_manifest" in globals() and batch_evaluation_manifest:
            print("A production-clean batch manifest already exists. No workshop override was needed.")
        else:
            if "batch_evaluation_manifest_before_workshop_override" not in globals():
                if "batch_result" not in globals():
                    raise RuntimeError("Run Step 15 before using the workshop batch baseline override.")
                print("Recovering the workshop override record from the existing batch_result.")
                batch_evaluation_manifest_before_workshop_override = {
                    "version": "1.0",
                    "artifact_type": "section_03_batch_evaluation_manifest",
                    "created_at": utc_now(),
                    "deployment_id": DEPLOYMENT_ID,
                    "dataset_lineage_id": DATASET_MANIFEST.get("dataset_lineage_id"),
                    "managed_datasets": DATASET_MANIFEST.get("managed_datasets"),
                    "source_section02": DATASET_MANIFEST.get("source_section02"),
                    "scenario_count": len(release_batch_dataset.scenarios) if "release_batch_dataset" in globals() else None,
                    "evaluator_ids": list(BATCH_EVALUATOR_IDS) if "BATCH_EVALUATOR_IDS" in globals() else [],
                    "batch_evaluation": summarize_batch_evaluation_result(batch_result),
                    "batch_events_count": len(batch_events) if "batch_events" in globals() and batch_events else None,
                    "batch_error_events": batch_error_events if "batch_error_events" in globals() else [],
                    "manifest_validation_status": "FAILED",
                    "manifest_validation_error": "Recovered from existing batch_result after Step 15 returned a non-production-clean batch status.",
                }

```

```

print("BATCH EVALUATION WORKSHOP CONTINUATION OVERRIDE")
print("The real Step 15 batch result was not production-clean:")
original_status = batch_evaluation_manifest_before_workshop_override["batch_evaluation"].get("status")
print(f"  original_status={original_status}")
print(f"  validation_error={batch_evaluation_manifest_before_workshop_override.get('manifest_validation_error')}")
print("We are marking the batch manifest as COMPLETED only so later workshop notebooks can run.")
print("Do not use this override as a production release approval.")

batch_evaluation_manifest = deepcopy(batch_evaluation_manifest_before_workshop_override)
batch_evaluation_manifest["workshop_continuation_override"] = True
batch_evaluation_manifest["original_batch_evaluation_status"] = original_status
batch_evaluation_manifest["batch_evaluation"]["original_status"] = original_status
batch_evaluation_manifest["batch_evaluation"]["status"] = "COMPLETED"
batch_evaluation_manifest["manifest_validation_status"] = "WORKSHOP_OVERRIDE"

save_json(batch_evaluation_manifest, BATCH_EVALUATION_MANIFEST_PATH)
%store batch_evaluation_manifest
print(f"Workshop continuation batch manifest saved to {BATCH_EVALUATION_MANIFEST_PATH}")

```

```

In [ ]: print("=" * 60)
print("DEPLOYMENT COMPLETE!")
print("=" * 60)

print(f"\nInfrastructure:")
print(f"  AgentCore Runtime: {RUNTIME_ARN}")
print(f"  AgentCore Gateway: {GATEWAY_URL}")
print(f"  RBAC Interceptor: {INTERCEPTOR_ARN}")
print(f"  Cognito Pool: {USER_POOL_ID}")

print(f"\nMCP Tools (11 total):")
print(f"  Read tools (6): search_products, get_product_details, check_inventory,")
print(
    f"                  get_product_recommendations, compare_products, get_return_policy"
)
print(f"  Admin tools (5): create_product, update_product, delete_product,")
print(f"                  update_inventory, update_pricing")

print(f"\nRBAC:")
print(f"  Customer ({redact_email(CUSTOMER_EMAIL)}): 6 read-only tools")
print(f"  Admin ({redact_email(ADMIN_EMAIL)}): 11 tools (6 read + 5 admin)")
print(f"  Enforcement: Agent-side filtering + Gateway interceptor")

print(f"\nObservability:")
print(f"  OTEL Service Name: {OTEL_SERVICE_NAME}")
print(f"  Traces: X-Ray via OTLP")
print(f"  GenAI Events: CloudWatch vendedlogs")
if "batch_evaluation_manifest" in globals():
    print(f"  Batch Evaluation: {batch_evaluation_manifest['batch_evaluation']['batch_evaluation_id']}")
print(
    f"  Dashboard: https://{REGION}.console.aws.amazon.com/cloudwatch/home?region={REGION}#genai-observability:bedrockAgentCore"
)

print("=" * 60)

```

```

In [ ]: from deployment_contract import (
        BATCH_EVALUATION_MANIFEST_PATH,
        build_deployment_manifest,
        load_json,
        save_json,
    )

# Save deployment info for subsequent modules (Module 04: Evaluations)
deployment_info = {
    "deployment_id": DEPLOYMENT_ID,
    "deployment_manifest_path": str(DEPLOYMENT_MANIFEST_PATH.relative_to(REPO_ROOT)),
    "batch_evaluation_manifest_path": str(BATCH_EVALUATION_MANIFEST_PATH.relative_to(REPO_ROOT)),
    "runtime_arn": RUNTIME_ARN,
    "runtime_id": RUNTIME_ID,
    "runtime_name": RUNTIME_NAME,
    "gateway_id": GATEWAY_ID,
    "gateway_url": GATEWAY_URL,
    "user_pool_id": USER_POOL_ID,
    "user_client_id": USER_CLIENT_ID,
    "otel_service_name": OTEL_SERVICE_NAME,
    "region": REGION,
}

observability_summary = {
    "log_delivery": LOG_DELIVERY_CONFIG if "LOG_DELIVERY_CONFIG" in globals() else {},
    "data_protection": DATA_PROTECTION_STATUS if "DATA_PROTECTION_STATUS" in globals() else {"status": "NOT_RUN"},
    "total_observability_events": total_data if "total_data" in globals() else None,
    "custom_spans_expected": sorted(EXPECTED_CUSTOM_SPANS) if "EXPECTED_CUSTOM_SPANS" in globals() else [],
}

image_evidence = IMAGE_EVIDENCE if "IMAGE_EVIDENCE" in globals() else {
    "repository": ECR_REPO_NAME,
    "tag": IMAGE_TAG if "IMAGE_TAG" in globals() else "latest",
    "image_uri": CONTAINER_URI,
    "digest": None,
}

if "batch_evaluation_manifest" in globals():
    batch_evaluation_record = batch_evaluation_manifest
elif BATCH_EVALUATION_MANIFEST_PATH.is_file():
    batch_evaluation_record = load_json(BATCH_EVALUATION_MANIFEST_PATH)
else:
    raise RuntimeError("Run Step 15 batch evaluation before saving deployment_manifest.json")

runtime_source_sha256 = hashlib.sha256(
    (SECTION_DIR / "agents" / "product_catalog_agent.py").read_bytes()
).hexdigest()

deployment_manifest = build_deployment_manifest(
    deployment_id=DEPLOYMENT_ID,
    region=REGION,
    account_id=ACCOUNT_ID,
    runtime={
        "runtime_id": RUNTIME_ID,
        "runtime_arn": RUNTIME_ARN,
    },

```

```

        "runtime_name": RUNTIME_NAME,
        "config_bundle_hook_version": "1",
        "config_bundle_source_sha256": runtime_source_sha256,
    },
    gateway={
        "gateway_id": GATEWAY_ID,
        "gateway_url": GATEWAY_URL,
        "gateway_arn": GATEWAY_ARN,
    },
    cognito={
        "user_pool_id": USER_POOL_ID,
        "user_client_id": USER_CLIENT_ID,
    },
    image=image_evidence,
    model_id=MODEL_ID,
    otel_service_name=OTEL_SERVICE_NAME,
    dataset_manifest=DATASET_MANIFEST,
    section02=SECTION02_QUALITY_CONTRACT,
    prompt_version=PROMPT_VERSION,
    tool_policy_version=TOOL_POLICY_VERSION,
    observability=observability_summary,
    quality_gate=postdeploy_quality_gate if "postdeploy_quality_gate" in globals() else {"status": "NOT_RUN"},
    batch_evaluation=batch_evaluation_record,
)

save_json(deployment_manifest, DEPLOYMENT_MANIFEST_PATH)
print(f"Deployment manifest saved to {DEPLOYMENT_MANIFEST_PATH}")

%store deployment_info
%store deployment_manifest
%store REGION
print("Deployment information saved for subsequent modules (Module 04: Evaluations)")

# Also store individual resource IDs for cleanup resilience
# (if kernel restarts, cleanup cell can recover these)
%store WORKSHOP_PREFIX
%store RUNTIME_ID
%store RUNTIME_ARN
%store RUNTIME_NAME
%store GATEWAY_ID
%store GATEWAY_URL
%store USER_POOL_ID
%store DEPLOYMENT_ID

# Save to JSON file as additional fallback
_deploy_file = "deployment_config.json"
with open(_deploy_file, "w") as _f:
    json.dump(deployment_info, _f, indent=2)
print(f"Also saved to {_deploy_file} (fallback for new sessions)")

```

Module 3 Summary

You deployed the Product Catalog Agent and produced deployment evidence.

The durable outputs are `deployment_manifest.json`, `batch_evaluation_manifest.json`, dataset lineage, runtime identifiers, image digest, trace evidence, and the post-deployment evaluation summary. Later notebooks use these artifacts to observe production behavior, update the dataset, and decide whether optimized variants are ready for release experiments.

Cleanup

Run cleanup only when you are finished with the workshop resources.

Cleanup deletes deployed infrastructure and should be treated as a deliberate teardown step. Keep the generated manifests if you want to preserve evidence of what was created and tested.

```
In [ ]: ## UNCOMMENT AND RUN TO CLEAN UP ALL RESOURCES

## --- Recover variables if kernel was restarted ---
# try:
#     %store -r WORKSHOP_PREFIX
#     %store -r REGION
#     %store -r RUNTIME_ID
#     %store -r GATEWAY_ID
#     %store -r USER_POOL_ID
#     print("Recovered variables from %store")
# except:
#     # Fallback: Load from JSON file
#     import json as _json
#     try:
#         with open('deployment_config.json') as _f:
#             _info = _json.load(_f)
#             REGION = _info.get('region', 'us-west-2')
#             RUNTIME_ID = _info.get('runtime_id')
#             GATEWAY_ID = _info.get('gateway_id')
#             USER_POOL_ID = _info.get('user_pool_id')
#             WORKSHOP_PREFIX = 'ecommerce-workshop'
#             print("Recovered variables from deployment_config.json")
#     except FileNotFoundError:
#         print("ERROR: No saved state found. Please run the setup cells first.")

# import boto3
# os.environ['AWS_REGION'] = REGION
# os.environ['AWS_DEFAULT_REGION'] = REGION
# lambda_client = boto3.client('Lambda', region_name=REGION)
# iam_client = boto3.client('iam', region_name=REGION)
# agentcore_client = boto3.client('bedrock-agentcore', region_name=REGION)
# agentcore_control_client = boto3.client('bedrock-agentcore-control', region_name=REGION)
# gateway_client = boto3.client('bedrock-agentcore-control', region_name=REGION)
# ecr_client = boto3.client('ecr', region_name=REGION)
# cognito_client = boto3.client('cognito-idp', region_name=REGION)

# from utils import delete_agent_runtime, delete_gateway

# print("Cleaning up Module 03 resources...\n")
```

```

## # 1. Delete AgentCore Runtime
# print("Deleting AgentCore Runtime...")
# delete_agent_runtime(agentcore_client, RUNTIME_ID)

## # 2. Delete AgentCore Gateway (and targets)
# print("\nDeleting AgentCore Gateway...")
# delete_gateway(gateway_client, GATEWAY_ID)

## # 3. Delete Lambda functions
# print("\nDeleting Lambda functions...")
# lambda_client = boto3.client('lambda', region_name=REGION)
# for func_name in [f'{WORKSHOP_PREFIX}-product-tools', f'{WORKSHOP_PREFIX}-rbac-interceptor']:
#     try:
#         lambda_client.delete_function(FunctionName=func_name)
#         print(f" Deleted: {func_name}")
#     except Exception as e:
#         print(f" Error deleting {func_name}: {sanitize_error(e)}")

## # 4. Delete Cognito User Pool
# print("\nDeleting Cognito User Pool...")
# try:
#     cognito_client.delete_user_pool_domain(Domain=DOMAIN_PREFIX, UserPoolId=USER_POOL_ID)
#     cognito_client.delete_user_pool(UserPoolId=USER_POOL_ID)
#     print(f" Deleted pool: {USER_POOL_ID}")
# except Exception as e:
#     print(f" Error: {sanitize_error(e)}")

## # 5. Delete ECR repository
# print("\nDeleting ECR repository...")
# try:
#     ecr_client.delete_repository(repositoryName=ECR_REPO_NAME, force=True)
#     print(f" Deleted: {ECR_REPO_NAME}")
# except Exception as e:
#     print(f" Error: {sanitize_error(e)}")

## # 6. Delete IAM roles
# print("\nDeleting IAM roles...")
# for role_name in [LAMBDA_ROLE_NAME, GATEWAY_ROLE_NAME, RUNTIME_ROLE_NAME]:
#     try:
#         # Detach managed policies
#         policies = iam_client.list_attached_role_policies(RoleName=role_name)
#         for policy in policies['AttachedPolicies']:
#             iam_client.detach_role_policy(RoleName=role_name, PolicyArn=policy['PolicyArn'])
#         # Delete inline policies
#         inline = iam_client.list_role_policies(RoleName=role_name)
#         for policy_name in inline['PolicyNames']:
#             iam_client.delete_role_policy(RoleName=role_name, PolicyName=policy_name)
#         iam_client.delete_role(RoleName=role_name)
#         print(f" Deleted: {role_name}")
#     except Exception as e:
#         print(f" Error deleting {role_name}: {sanitize_error(e)}")

# print("\nCleanup complete!")

```